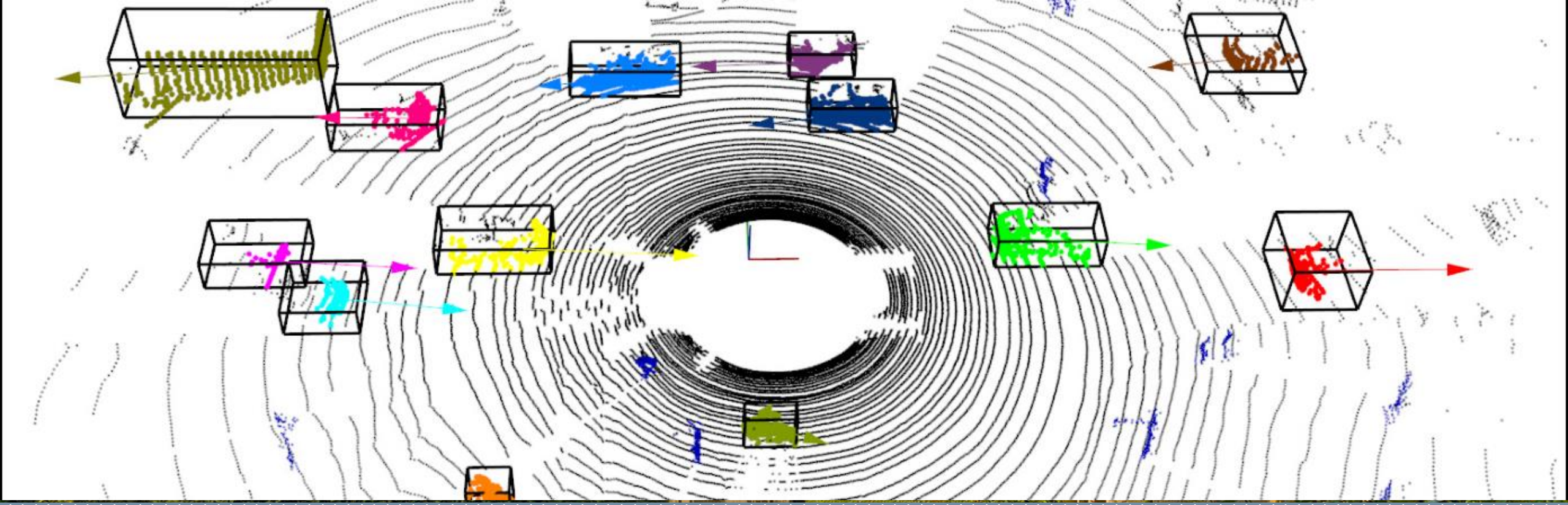


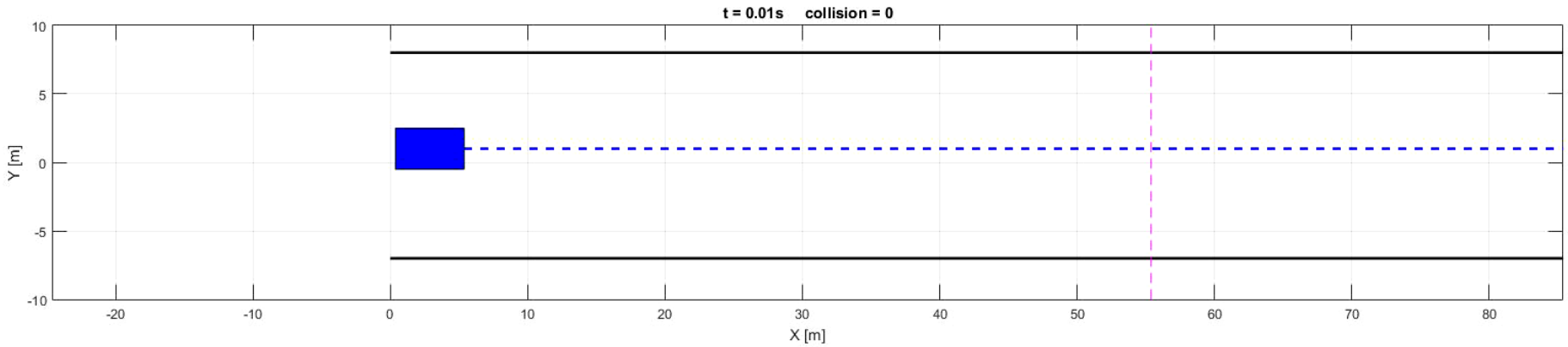
Automation and Control in Autonomous Vehicles (ACAV)

Target Tracking

Matteo Corno

Local planning requires information on:

- Obstacle position
- Obstacle velocity (direction and speed)



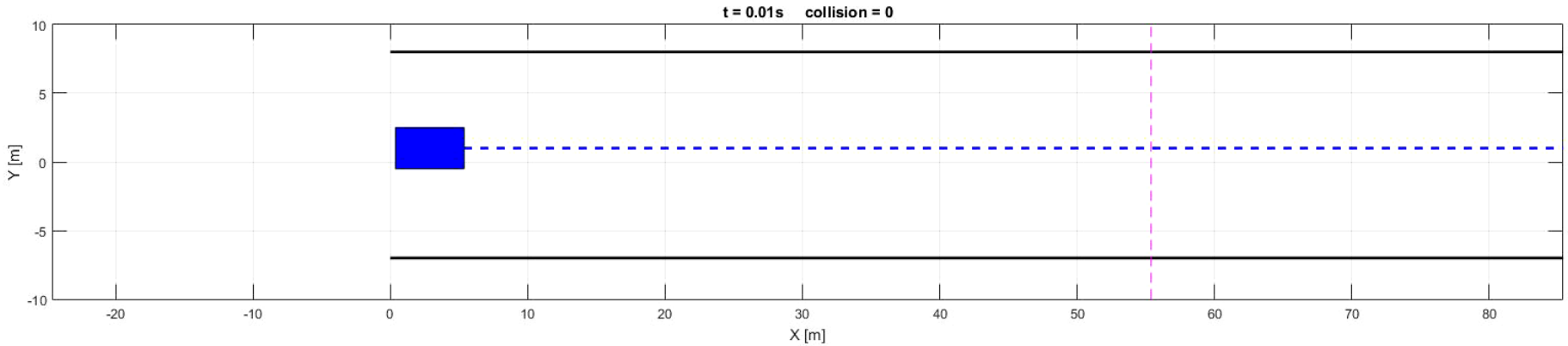
Introduction and Motivation

Automotive Applications

Local planning requires information on:

- Obstacle position
- Obstacle velocity (direction and speed)

} Dynamic problem in nature.



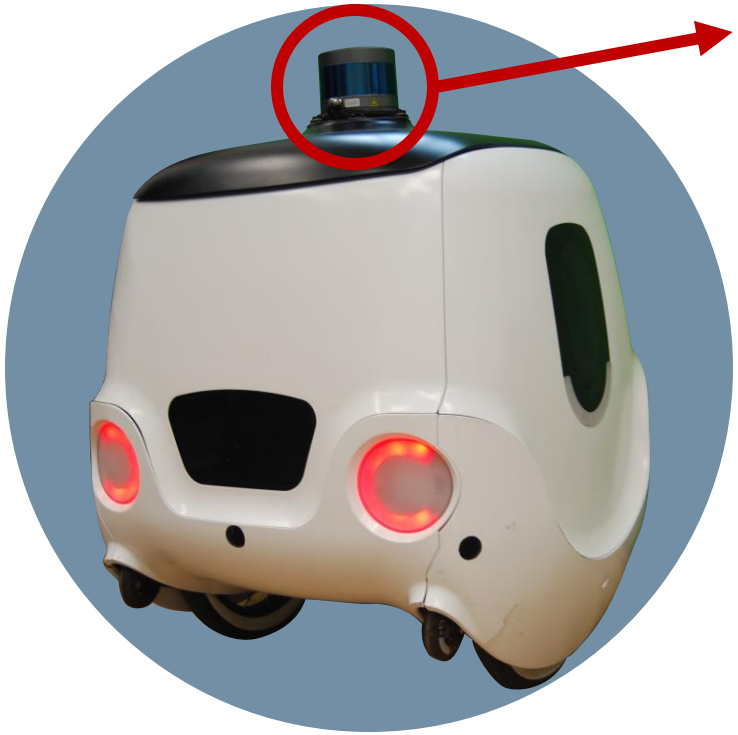
Local planning requires information on:

- Obstacle position
- Obstacle velocity (direction and speed)

} Dynamic problem in nature.

- In obstacle detection we can use only the point cloud
- To estimate velocity, we need to keep *track* of the obstacle over time

Point Cloud

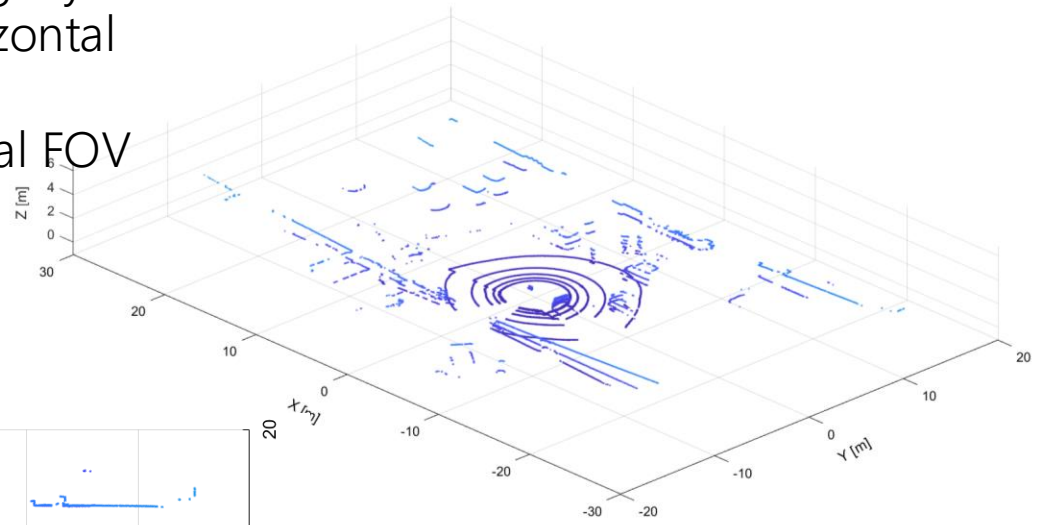


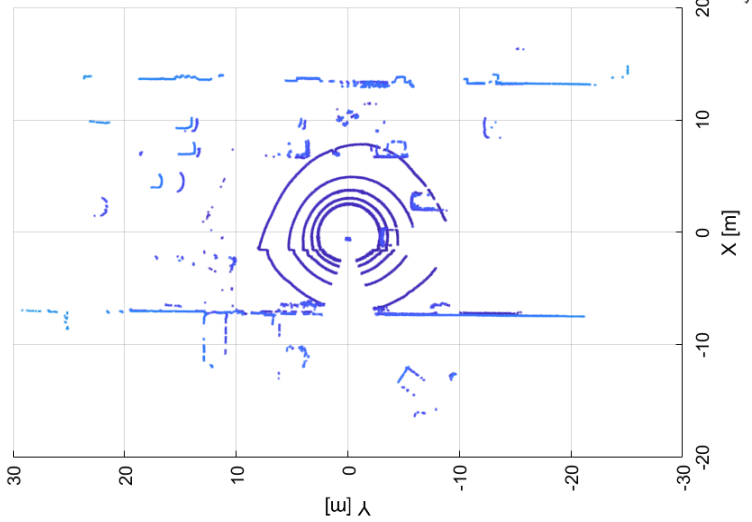


Quanergy M8

3D Mechanical LiDAR

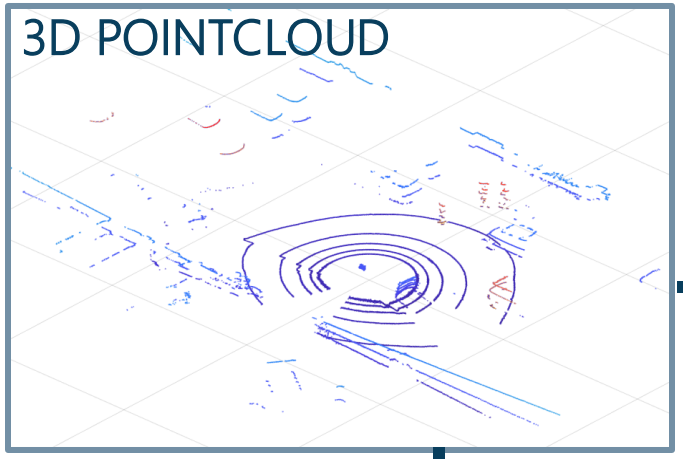
- 8 scanning layers
- 360° Horizontal FOV
- 20° Vertical FOV





Pointcloud processing

3D POINTCLOUD



2nd CONSIDERATION

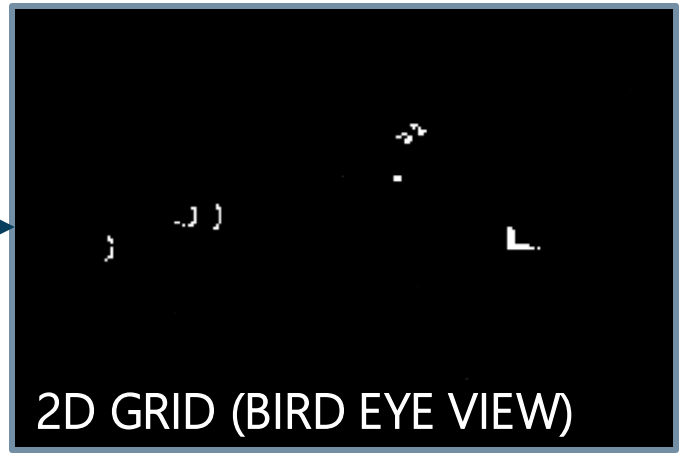
Z-coordinate is not very informative (no substantial difference in target height)

1st CONSIDERATION

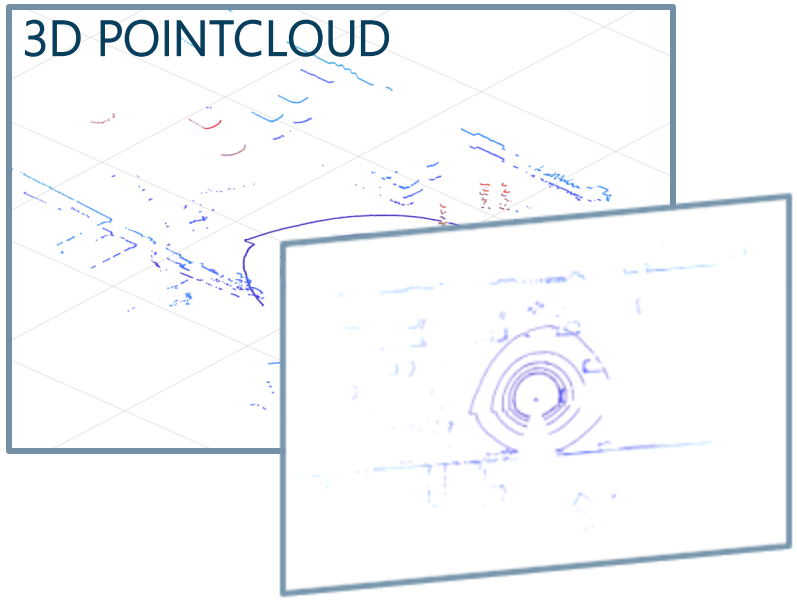
Ego and target vehicles motion is mostly planar

THE GOAL

Convert the input to reduce complexity



Pointcloud processing



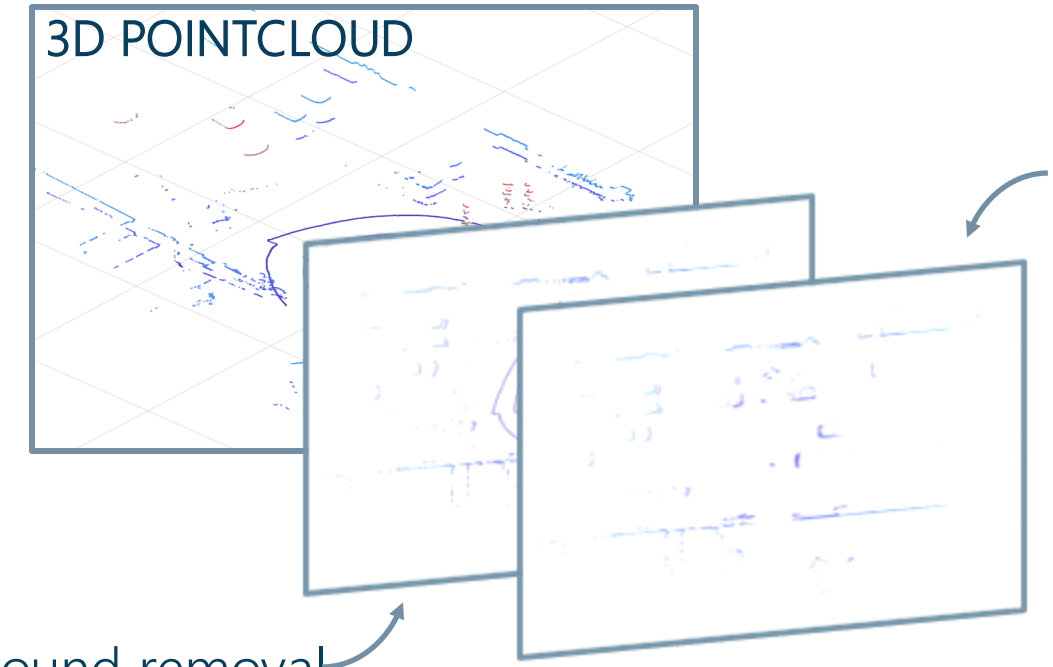
Ground removal

Ground Removal can be performed with RANSAC

Algorithm 15.4: RANSAC: fitting lines using random sample consensus

Determine:
  S — the smallest number of points required
  N — the number of iterations required
  d — the threshold used to identify a point that fits well
  T — the number of nearby points required
    to assert a model fits well
Until **N** iterations have occurred
  Draw a sample of **S** points from the data
    uniformly and at random
  Fit to that set of **S** points
  For each data point outside the sample
    Test the distance from the point to the line
      against **d** if the distance from the point to the line
      is less than **d** the point is close
  end
  If there are **T** or more points close to the line
    then there is a good fit. Refit the line using all
    these points.
end
Use the best fit from this collection, using the
  fitting error as a criterion

Pointcloud processing



- The target velocity estimation requires more frames.
- Yape moves because of the stabilization algorithm

- Scans taken at different time instants need to be realigned.
- The localization algorithm used for navigation is not accurate enough
- → Iterative Closest Point (ICP) approach

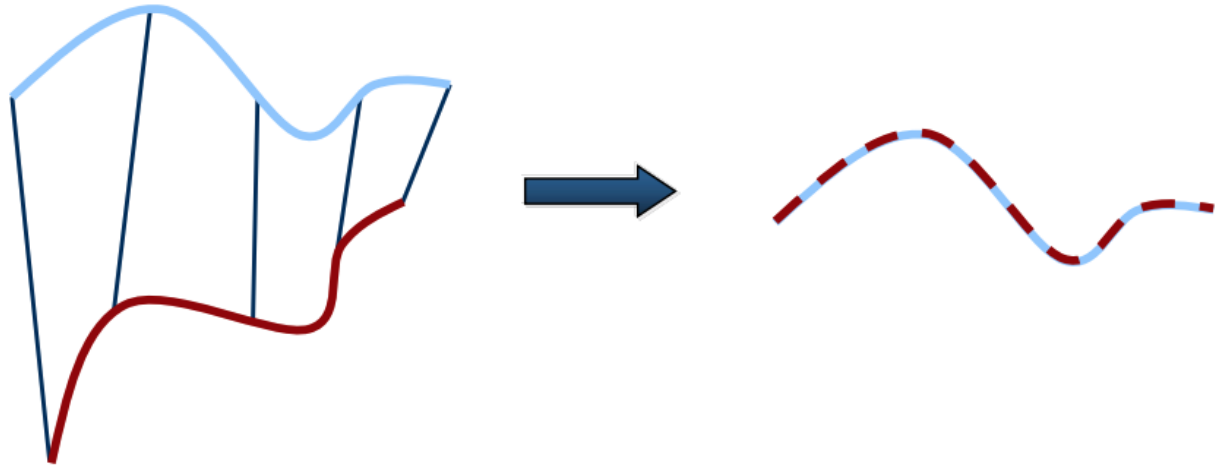
The Iterative Closest Point (ICP)

The problem:

- Given two point sets:

$$X = \{x_1, x_2, ..., x_n\}$$

$$P = \{p_1, p_2, ..., p_n\}$$



- Find a translation and a rotation that minimizes

$$E(R,t) = \frac{1}{N_p} \sum_{i=1}^{N_p} ||x_i - Rp_i - t||^2$$

where x_i and p_i are corresponding points

The Iterative Closest Point (ICP)

The algorithm needs to solve two separate steps:

1. Find the correspondence
2. Find the roto-translation

The Iterative Closest Point (ICP)

The algorithm needs to solve two separate steps:

1. Find the correspondence
2. **Find the roto-translation**

If we assume that the point correspondence is already known

1. Subtract the center of mass from every point in the two point

$$X' = \{x_i - \mu_x\} \qquad \mu_x = \frac{1}{N_x}\sum_{i=1}^{N_x} x_i$$

$$P' = \{p_i - \mu_p\} \qquad \mu_p = \frac{1}{N_p}\sum_{i=1}^{N_p} p_i$$

The Iterative Closest Point (ICP)

The algorithm needs to solve two separate steps:

1. Find the correspondence
2. **Find the roto-translation**

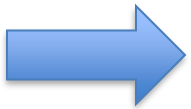
If we assume that the point correspondence is already known

2. The optimal solution of the optimization problem is obtained by setting

$$W = \sum_{i=1}^{N_p} x'_i p'^T_i$$

$$[U, V, \Sigma] = SVD(W)$$

$$W = U \begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & \sigma_3 \end{bmatrix} V$$



$$R = UV^T$$

$$t = \mu_x - R\mu_p$$

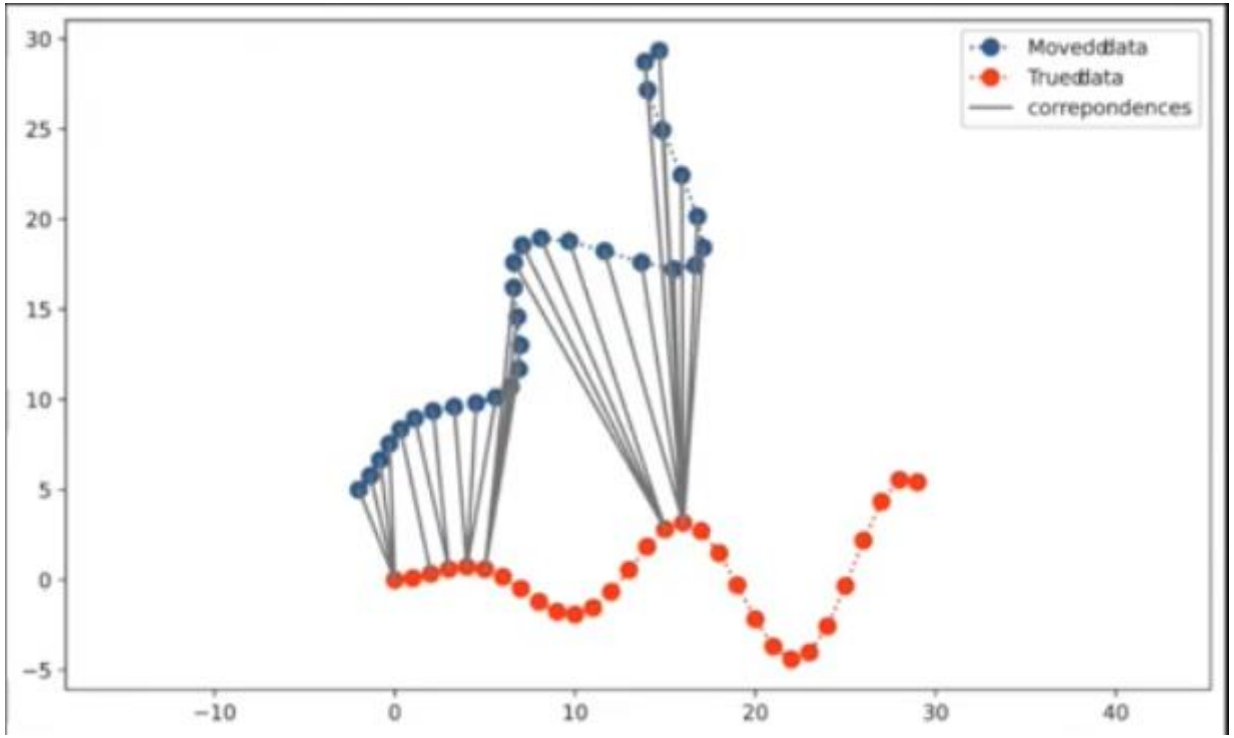
The Iterative Closest Point (ICP)

The algorithm needs to solve two separate steps:

1. **Find the correspondence**
2. Find the roto-translation

The correspondence is found iteratively

1. Find of each point the closest one on the other point cloud
2. Find the roto-translation



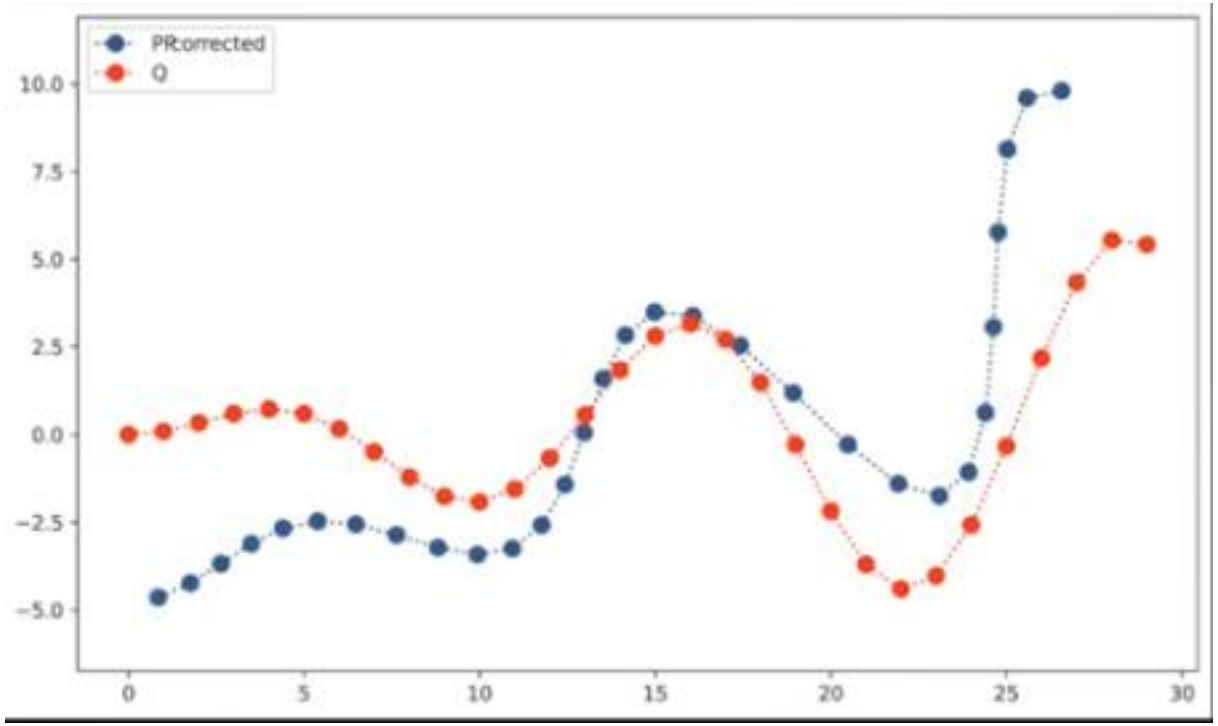
The Iterative Closest Point (ICP)

The algorithm needs to solve two separate steps:

1. **Find the correspondence**
2. Find the roto-translation

The correspondence is found iteratively

1. Find of each point the closest one on the other point cloud
2. Find the roto-translation
3. Apply the roto-translation



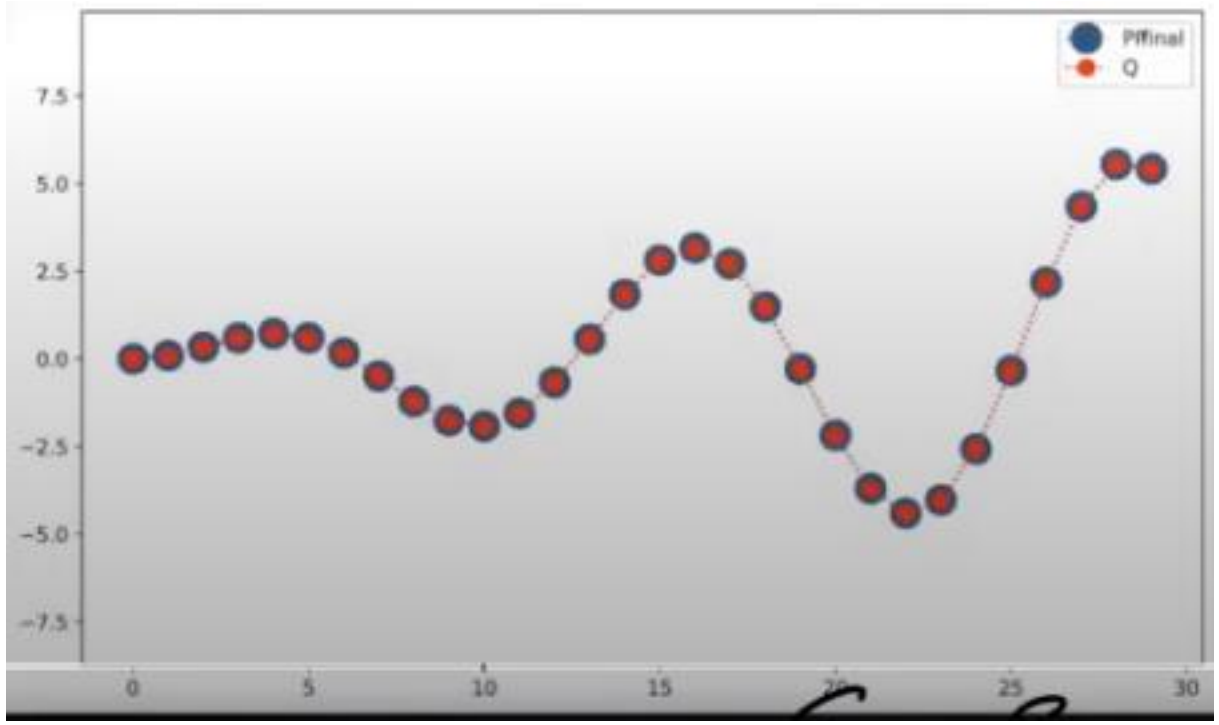
The Iterative Closest Point (ICP)

The algorithm needs to solve two separate steps:

1. **Find the correspondence**
2. Find the roto-traslation

The correspondence is found iteratively

1. Find of each point the closest one on the other point cloud
2. Find the roto-translation
3. Apply the roto-translation
4. Find a new point association

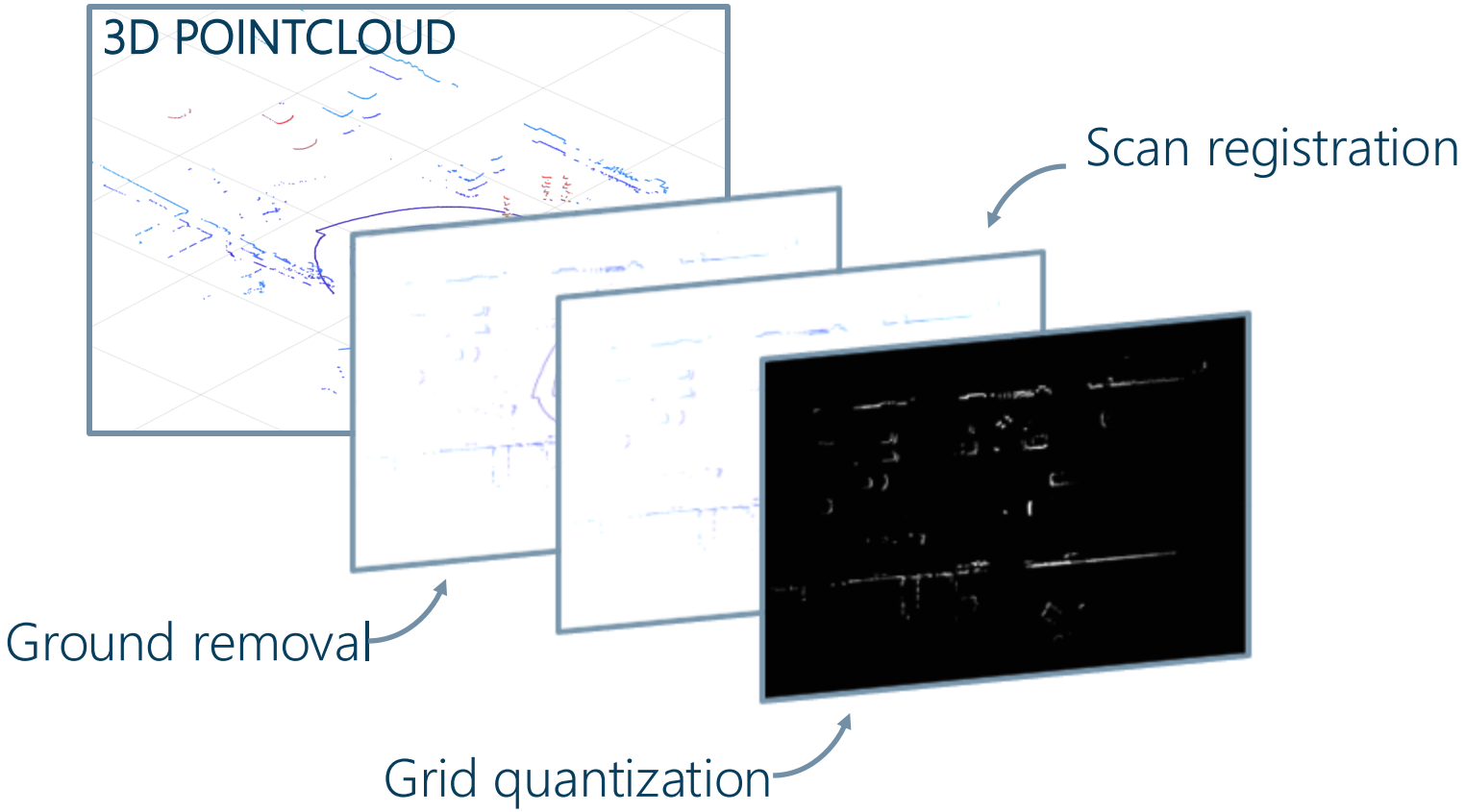


The Iterative Closest Point (ICP)

This is the basic algorithm → there are many variants that improve on the basic idea.

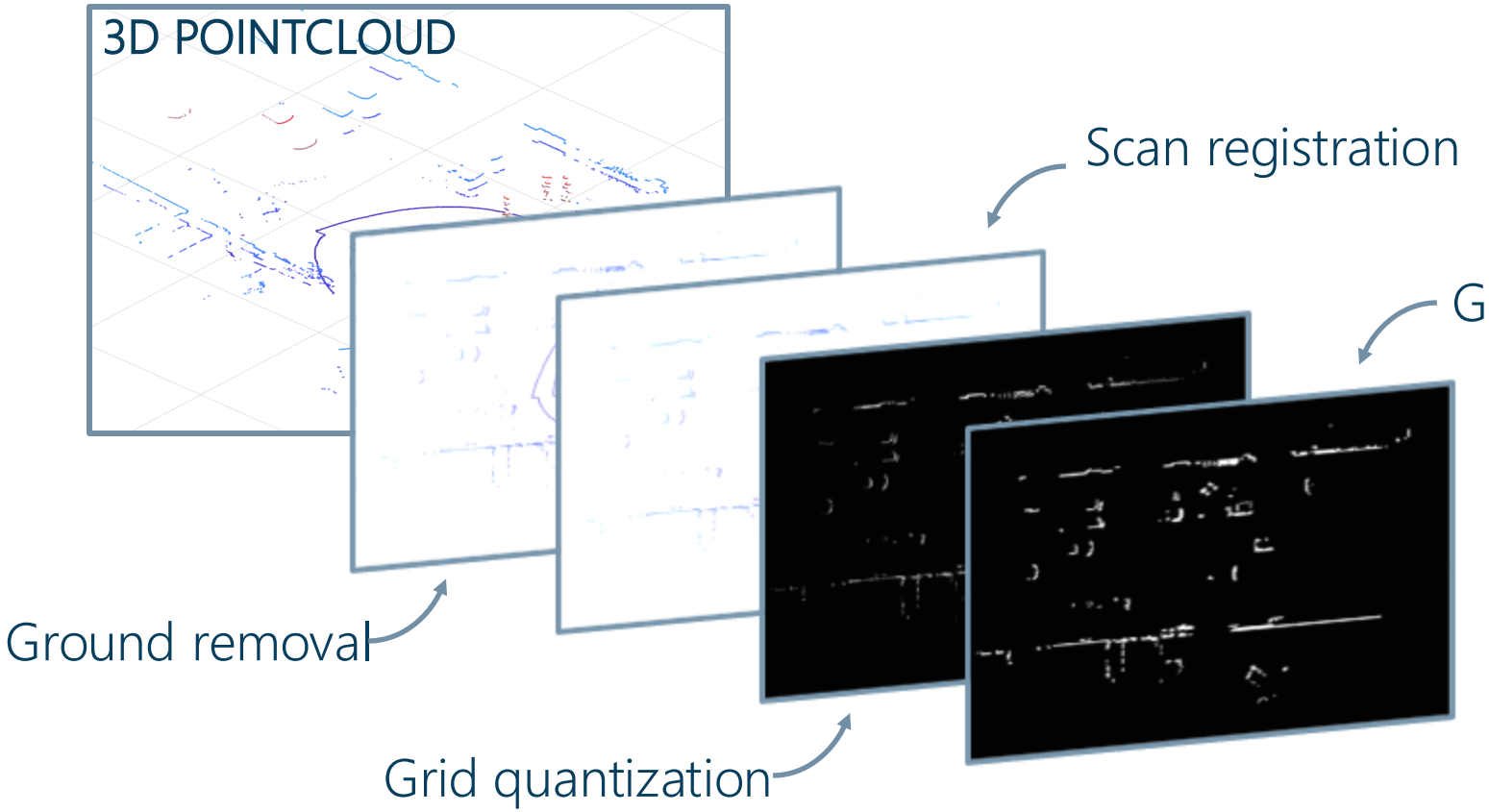
1. Point subsets (from one or both point sets)
2. Weighting the correspondences
3. Data association
4. Rejecting certain (outlier) point pairs

Pointcloud processing



- Grid overlay
- Each cell value is the number of points in it
- It improves the computational efficiency
- Lowers memory occupation efficiency
- The Lidar resolution in the z axis is low → z axis neglected

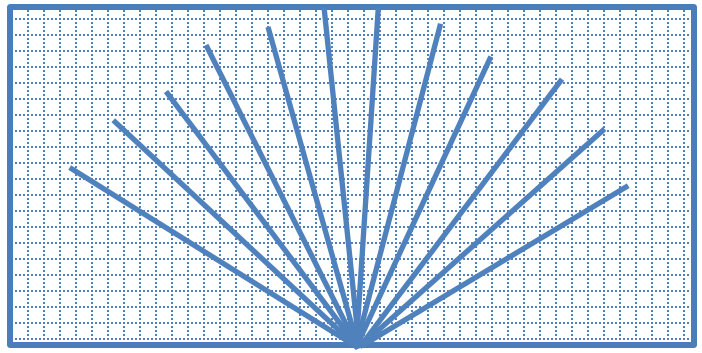
Pointcloud processing



- Each cell is given a 0 (free) – 1 (occupied) value.
- Constant threshold ?

Grid Binarization

- A constant threshold does not work
- The LiDAR does not sample the space uniformly.
- The point cloud becomes sparser as the distance from the sensor increases

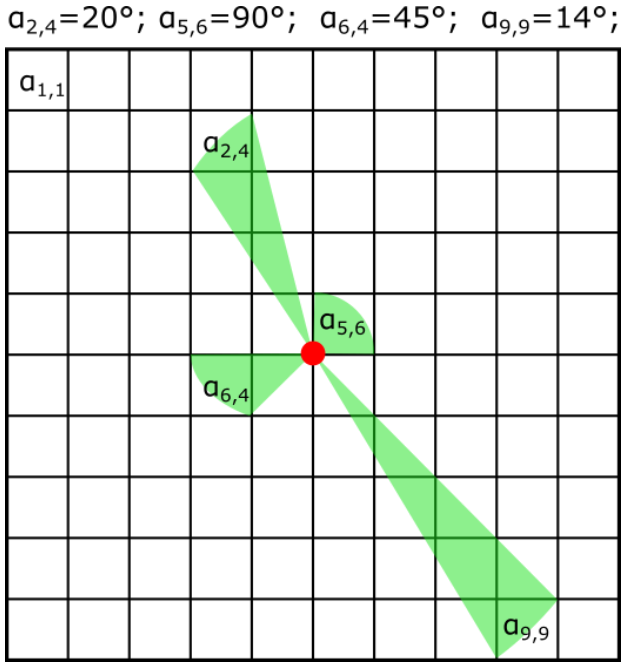


- The number of points in each cell is normalized by the maximum number the cell can contain

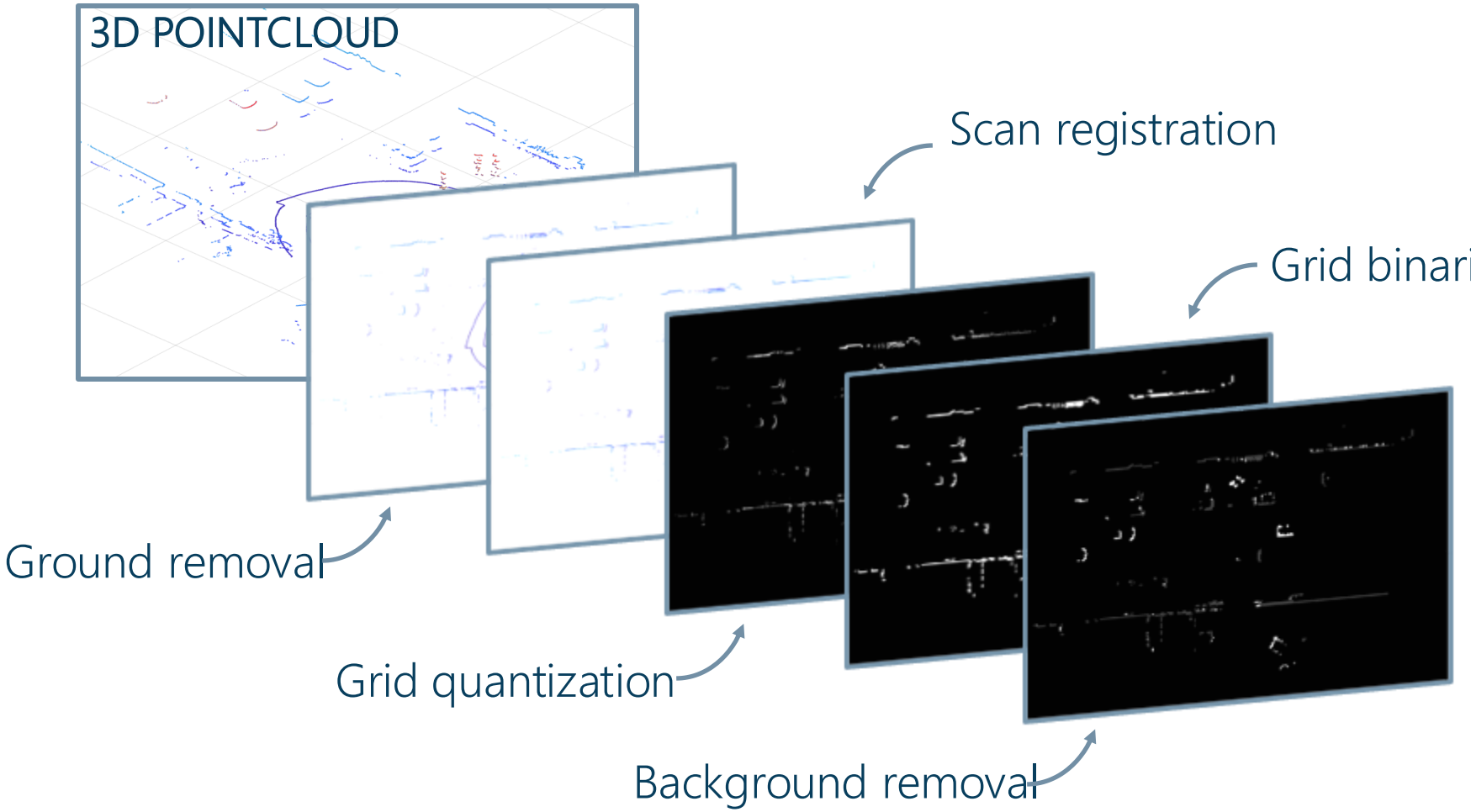
$$\bar{b}_{n,m} = \alpha(n,m)\delta$$

Angular Coverage of the cell

Angular point density of the LiDAR



Pointcloud processing



We need to separate the moving objects from the background.

Background Removal

The background image is estimated as

$$G(t) = \frac{\sum_{T=0}^{W} \tilde{B}(t-T)}{W}$$

The background image is computed by requiring:

$$\tilde{g}_{n,m} = \begin{cases} 1 \text{ if } g_{n,m} > \tau_{bck} \\ 0 \text{ otherwise} \end{cases} .$$

Background Removal

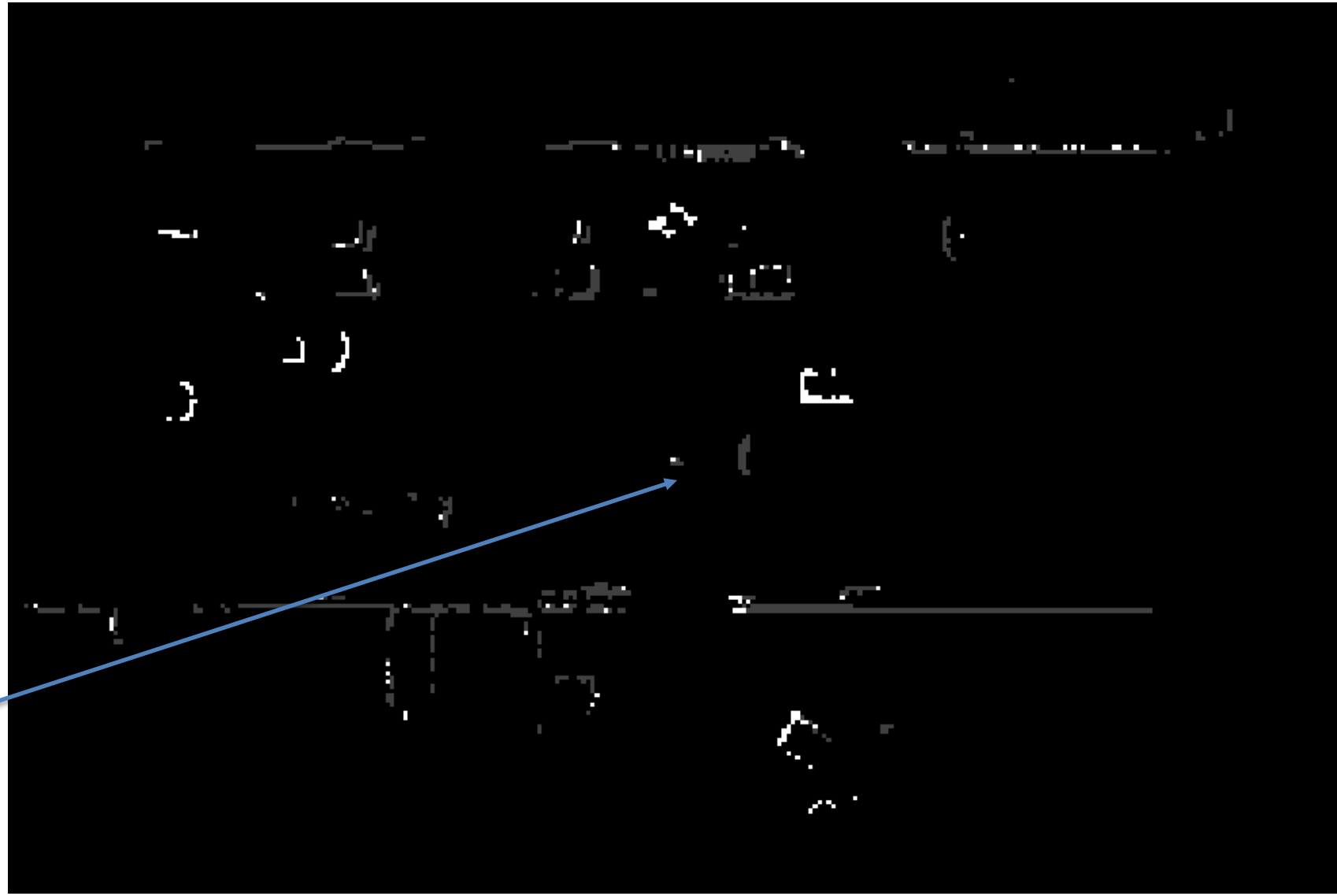
The background image is estimated as

$$G(t) = \frac{\sum_{T=0}^{W} \tilde{B}(t-T)}{W}$$

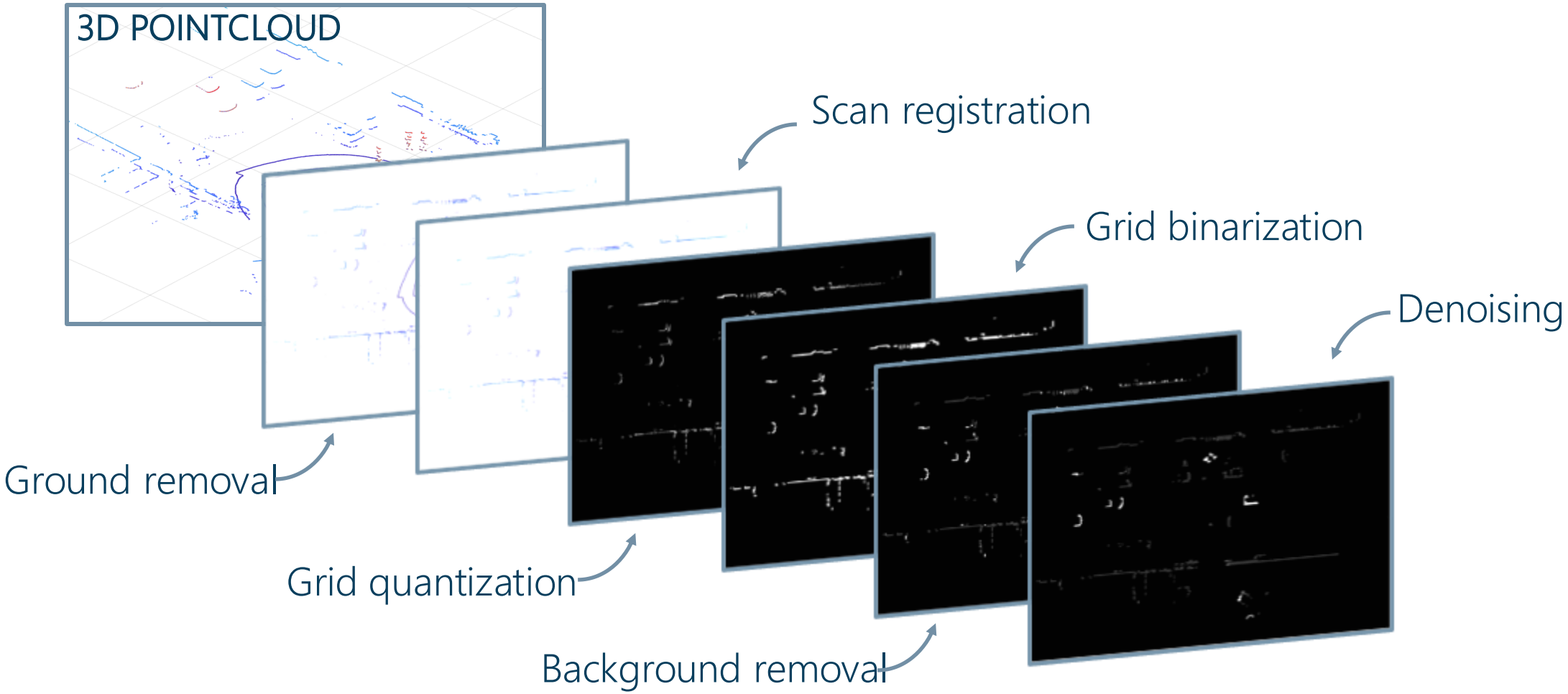
The background image is computed by requiring:

$$\tilde{g}_{n,m} = \begin{cases} 1 \text{ if } g_{n,m} > \tau_{bck} \\ 0 \text{ otherwise} \end{cases}$$

There is still some noise



Pointcloud processing



Denoising

Two step process:

1. Image processing
2. Shadow management

Denoising

Two step process:

1. **Image processing**
2. Shadow management

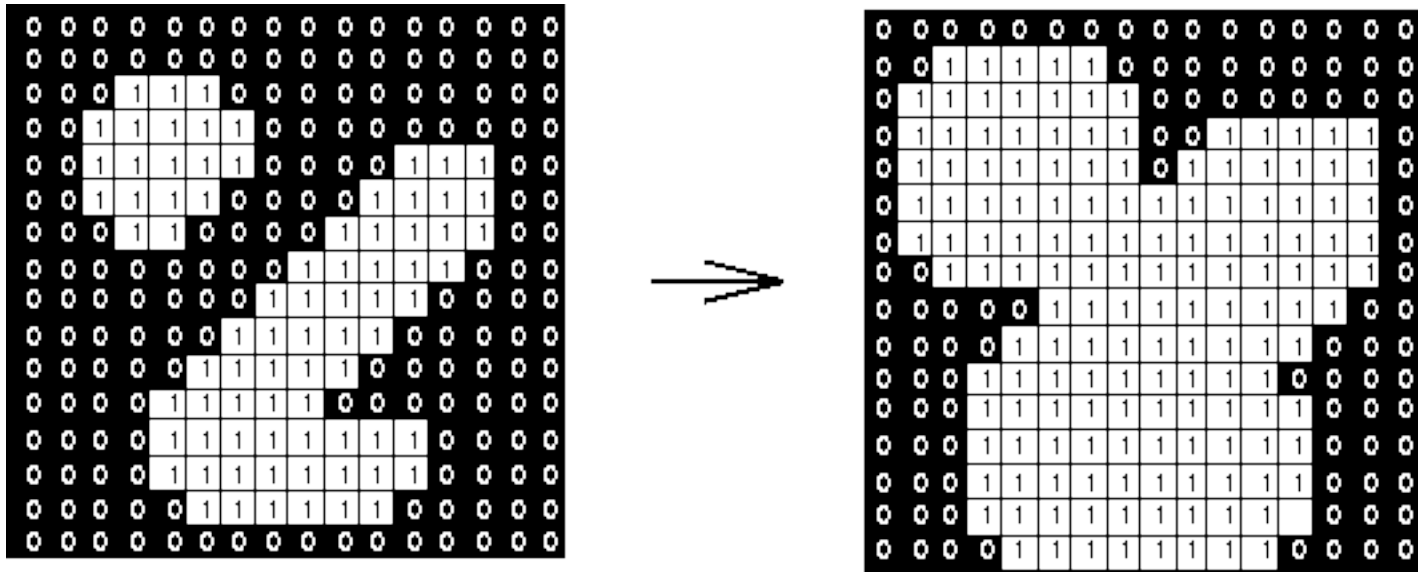
The goal of the image processing step is dual: it aims at removing noisy pixels and at aggregating as much as possible those pixels which are related to the same obstacle.

1. Background dilation
2. Closure to aggregate the sparse points belonging to a same obstacle.
3. Area thresholding.

Dilation

The goal of the image processing step is dual: it aims at removing noisy pixels and at aggregating as much as possible those pixels which are related to the same obstacle.

1. **Background dilation**
2. Closure to aggregate the sparse points belonging to a same obstacle.
3. Area thresholding.



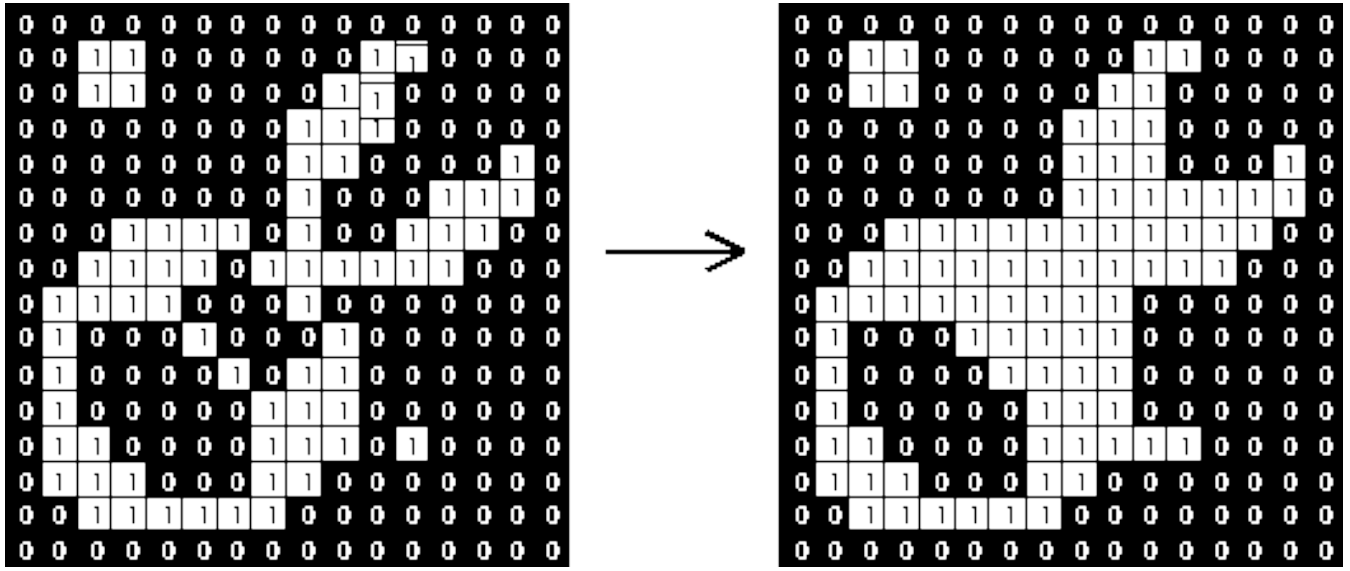
Dilation of a 3x3

Dilation

The goal of the image processing step is dual: it aims at removing noisy pixels and at aggregating as much as possible those pixels which are related to the same obstacle.

1. Background dilation
2. **Closure to aggregate the sparse points belonging to a same obstacle.**
3. Area thresholding.

Dilation + erosion



3x3 structuring element

Denoising

Two step process:

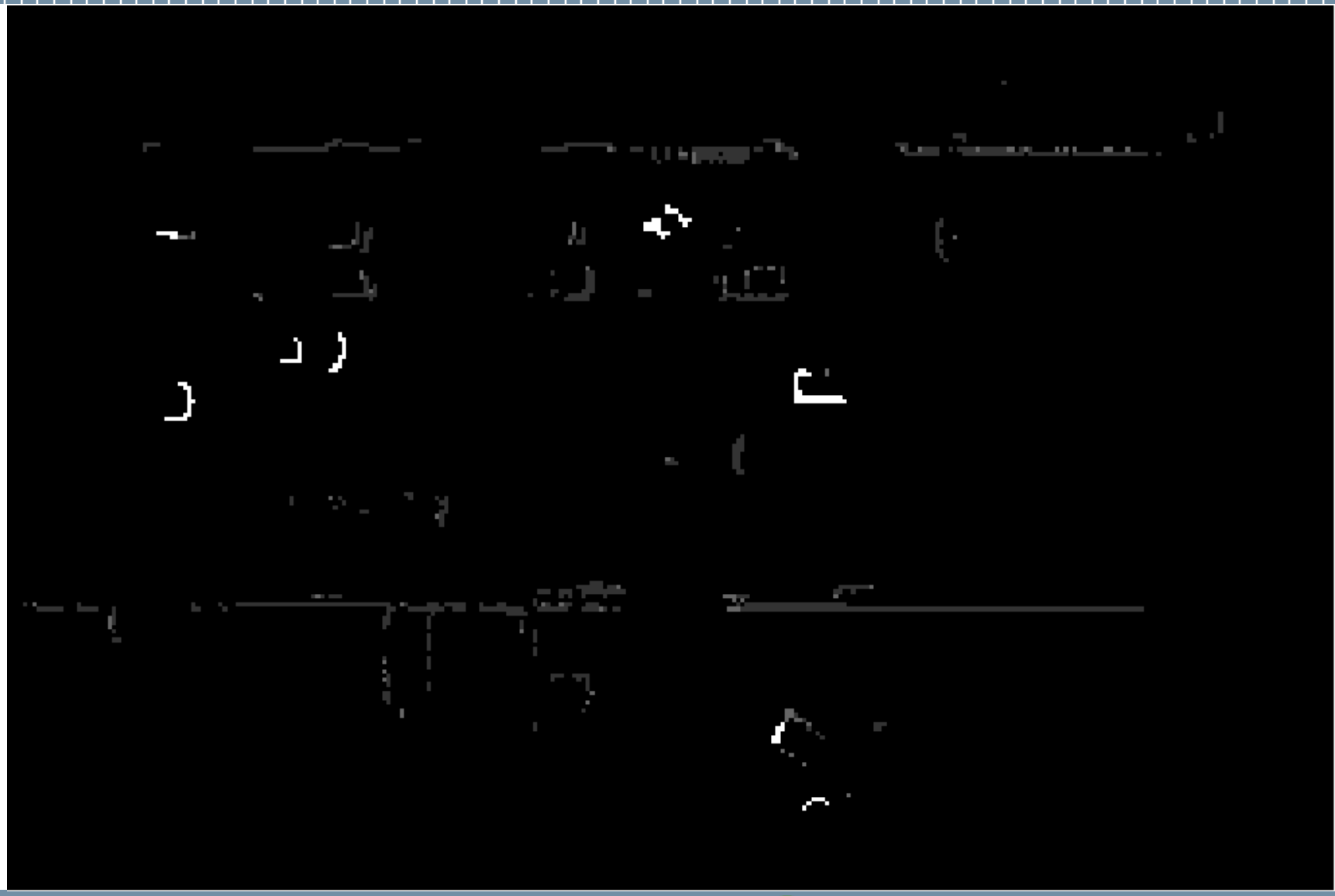
1. **Image processing**
2. Shadow management



Denoising

Two step process:

1. **Image processing**
2. Shadow management



Denoising

Two step process:

1. Image processing
2. **Shadow management**

Estimated shadow cones are created for each dynamical object detected in the scene. T

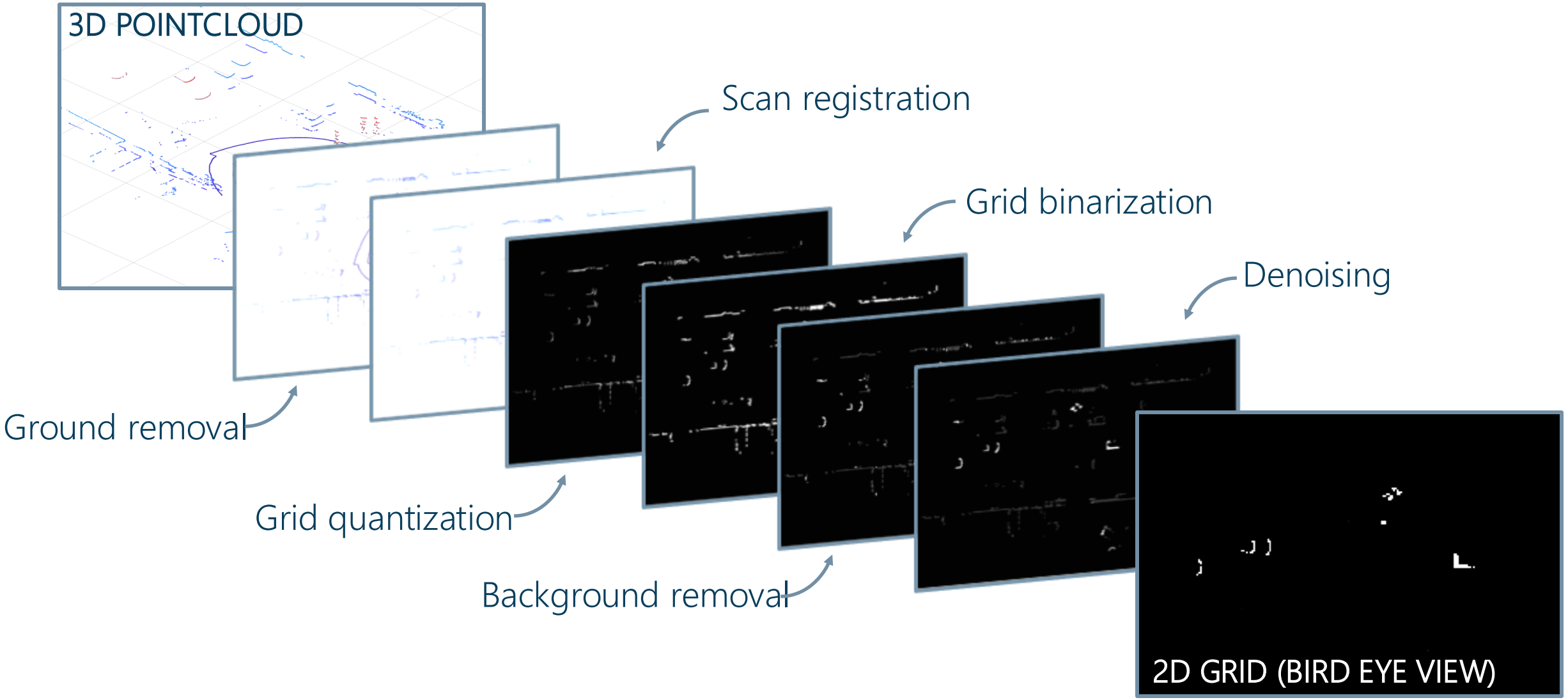
Shadowed pixels are kept constant until they are visible again.

Shadowed background is assumed not to change during the passage of the shadow cone.

It is an iterative approach.

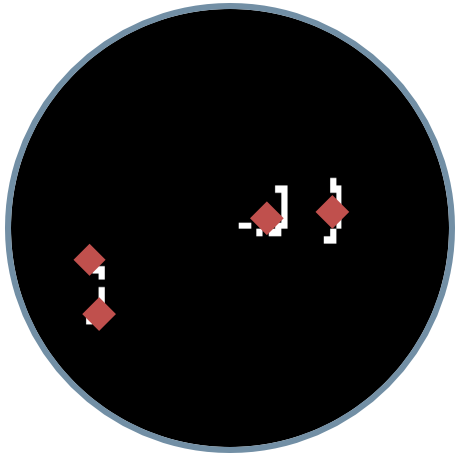


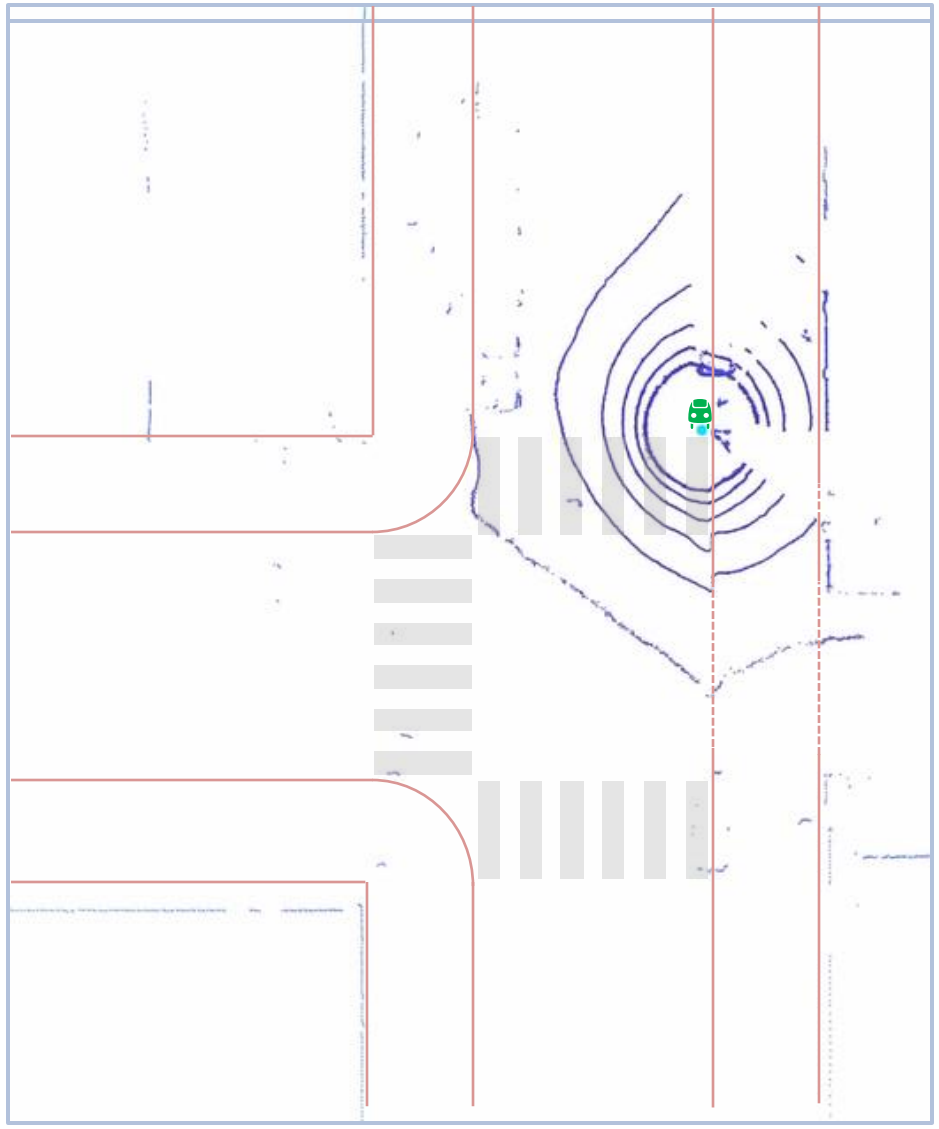
Pointcloud processing



Pointcloud processing

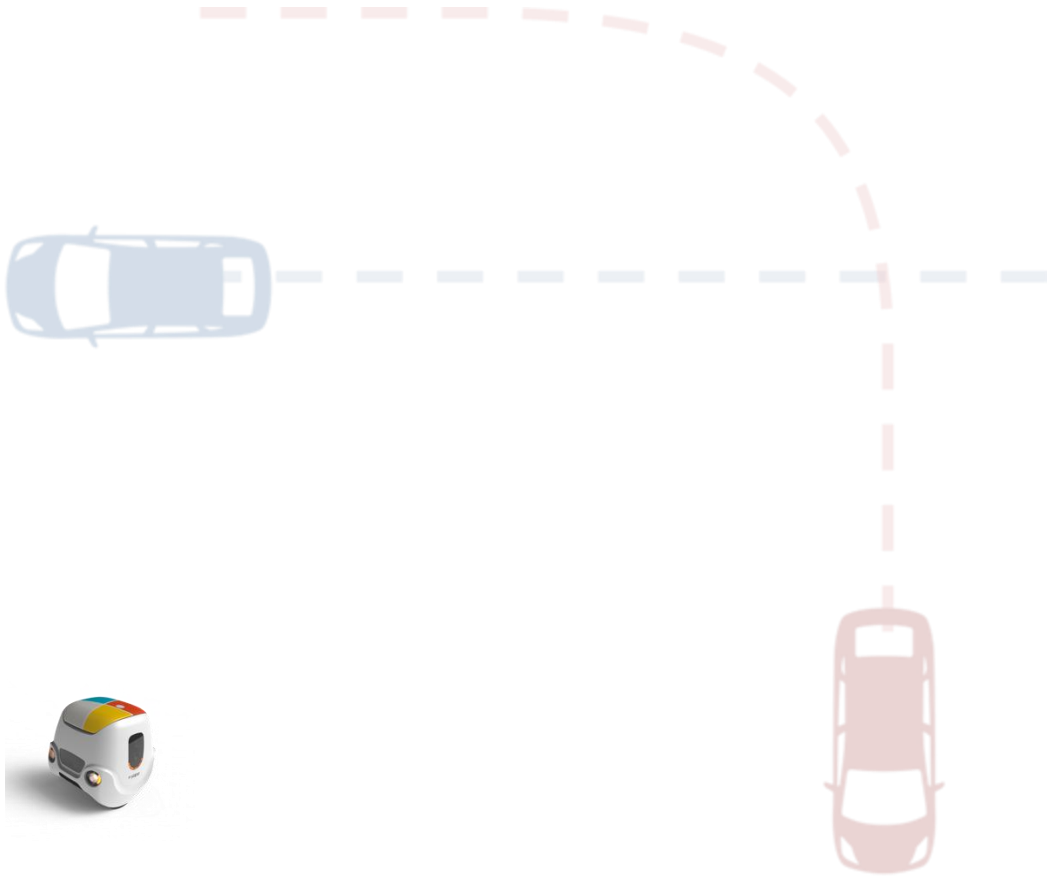
1. **Pointcloud Preprocessing**
   Ground removal: RANSAC plane fitting
   Scan registration: Iterative Closest Point
2. **Grid Quantization**
   Cell value equal to the number of 3D points in that patch
3. **Grid Binarization**
   Normalization and thresholding
4. **Static background removal**
   Pixelwise moving-average filtering
5. **Morphological operations**
   Noise removal by means of:
   - Background dilation
   - Image Closure
   - Area opening





Blobs geometrical centers are used as measures

Multi-Target Tracking

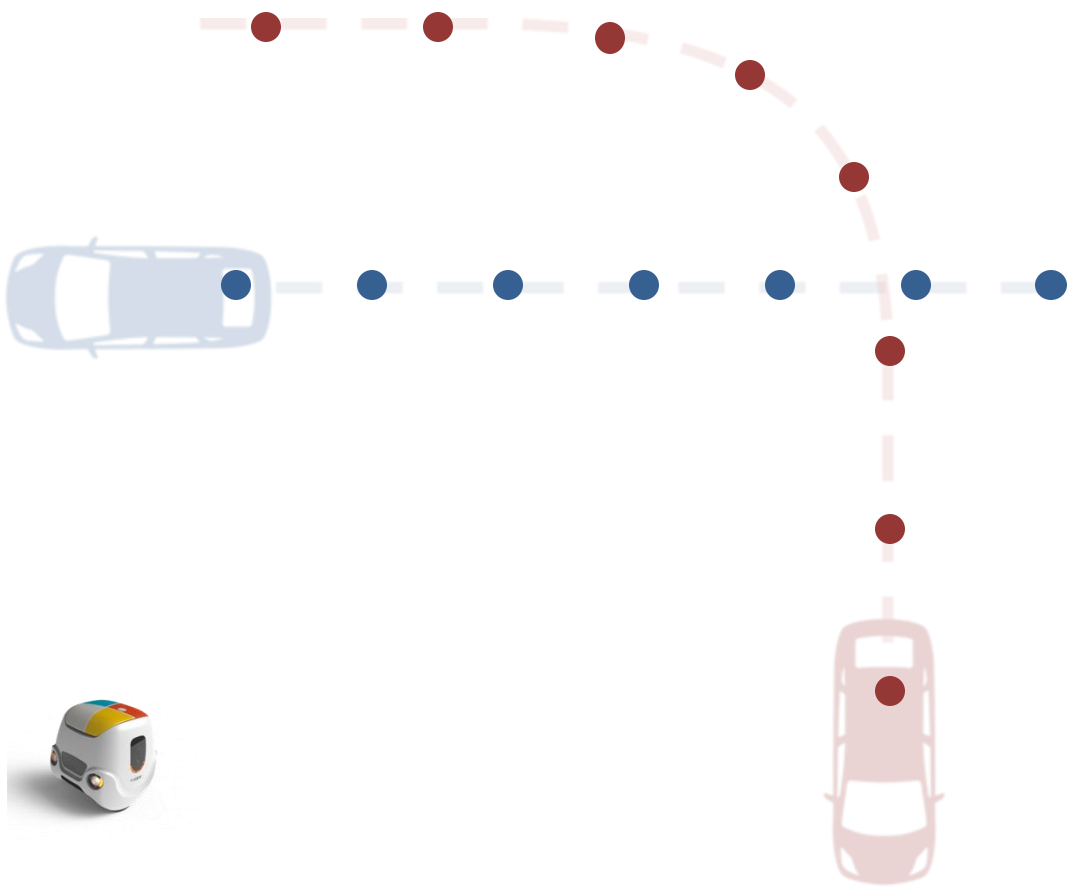


● Real position ◆ Measured position ⌖ Estimated position

Multi-Target Tracking:

1. Unknown number of targets
2. Each target generates at most one measure point

Multi-Target Tracking

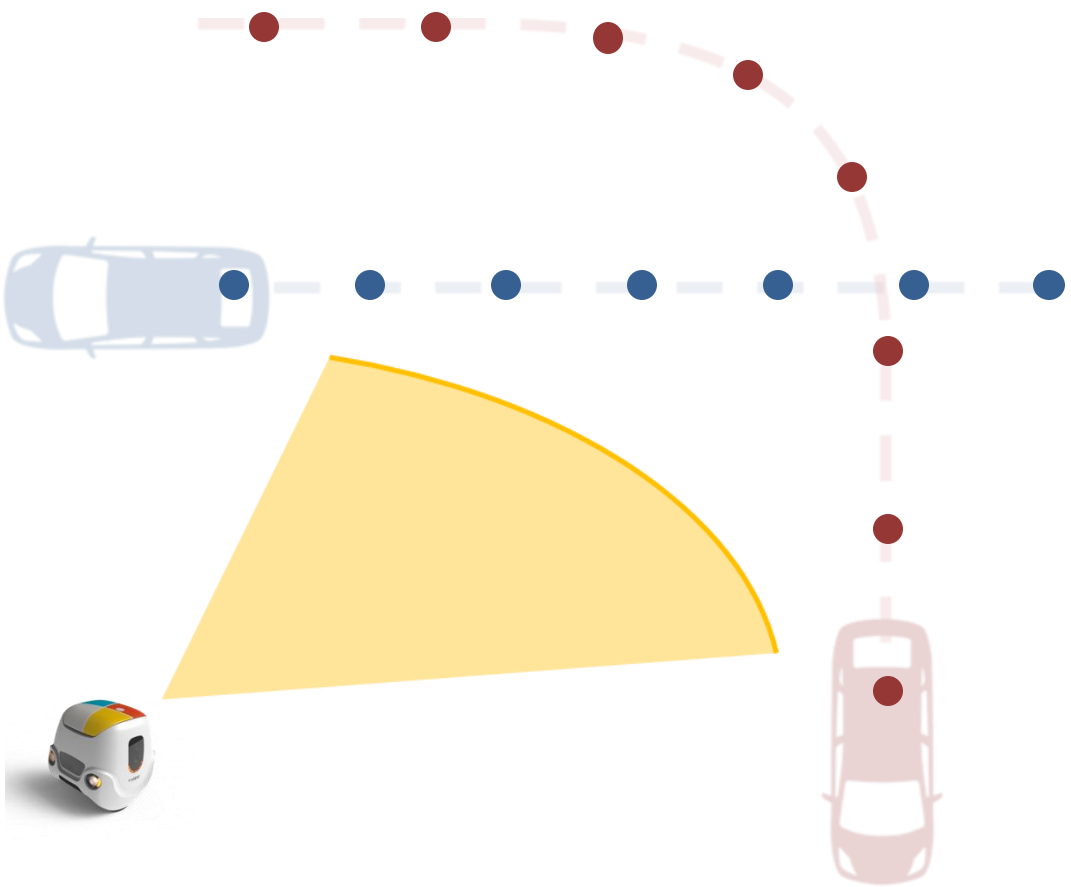


● Real position ◆ Measured position ⌖ Estimated position

Multi-Target Tracking:

1. Unknown number of targets
2. Each target generates at most one measure point

Multi-Target Tracking

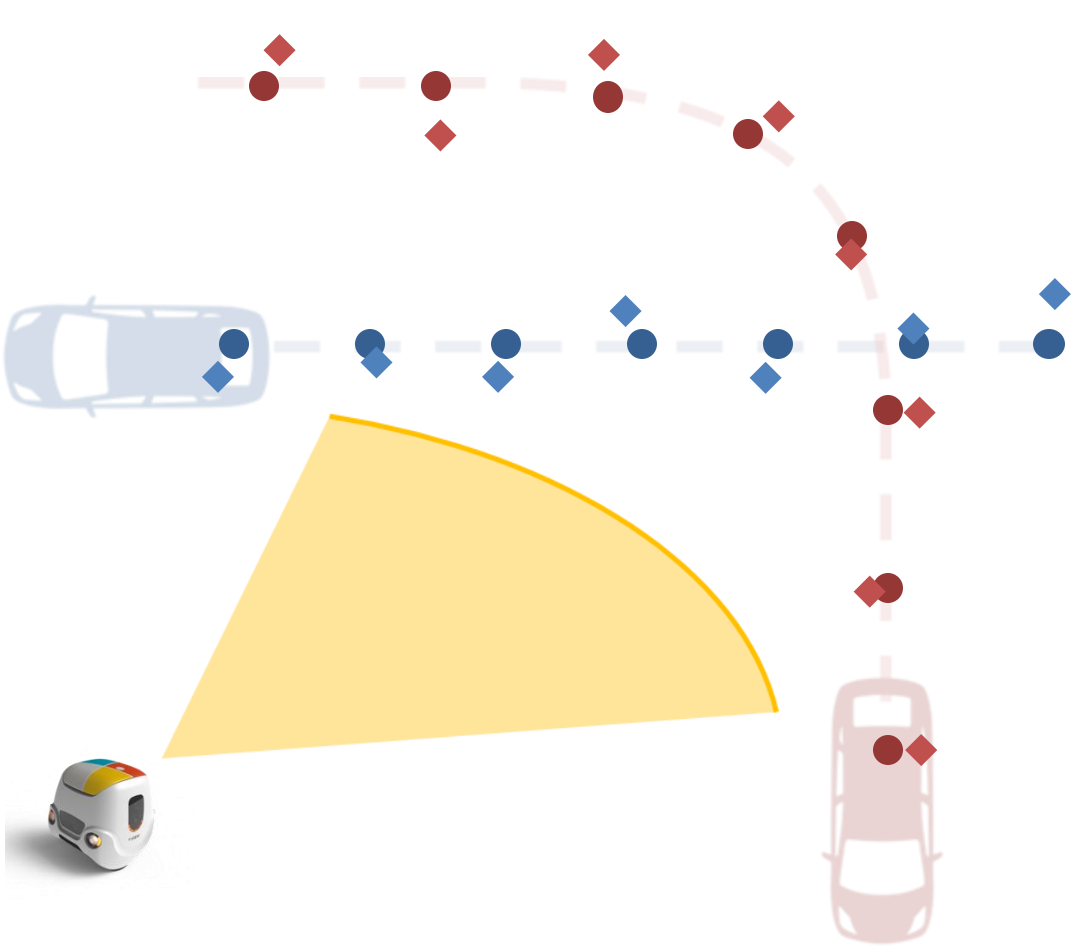


● Real position ◆ Measured position ⌖ Estimated position

Multi-Target Tracking:

1. Unknown number of targets
2. Each target generates at most one measure point

Multi-Target Tracking

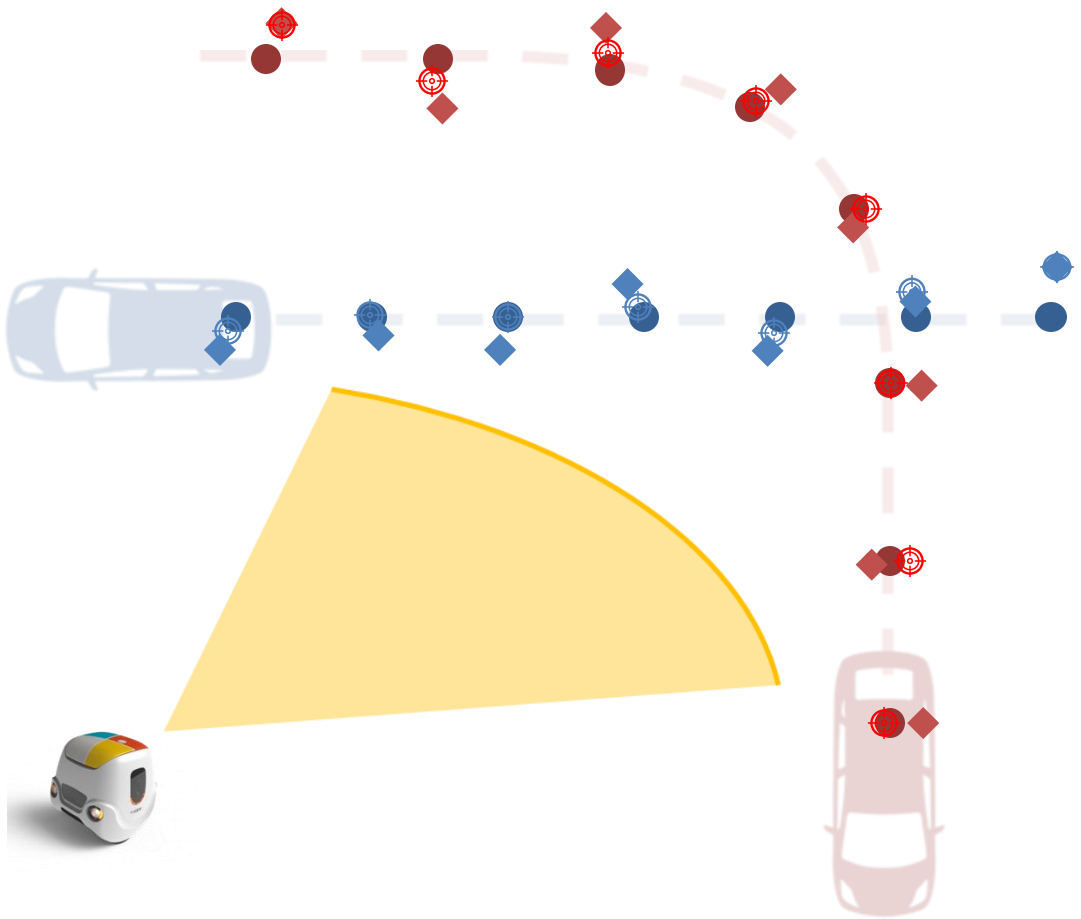


● Real position ◆ Measured position ⌖ Estimated position

Multi-Target Tracking:

1. Unknown number of targets
2. Each target generates at most one measure point

Multi-Target Tracking

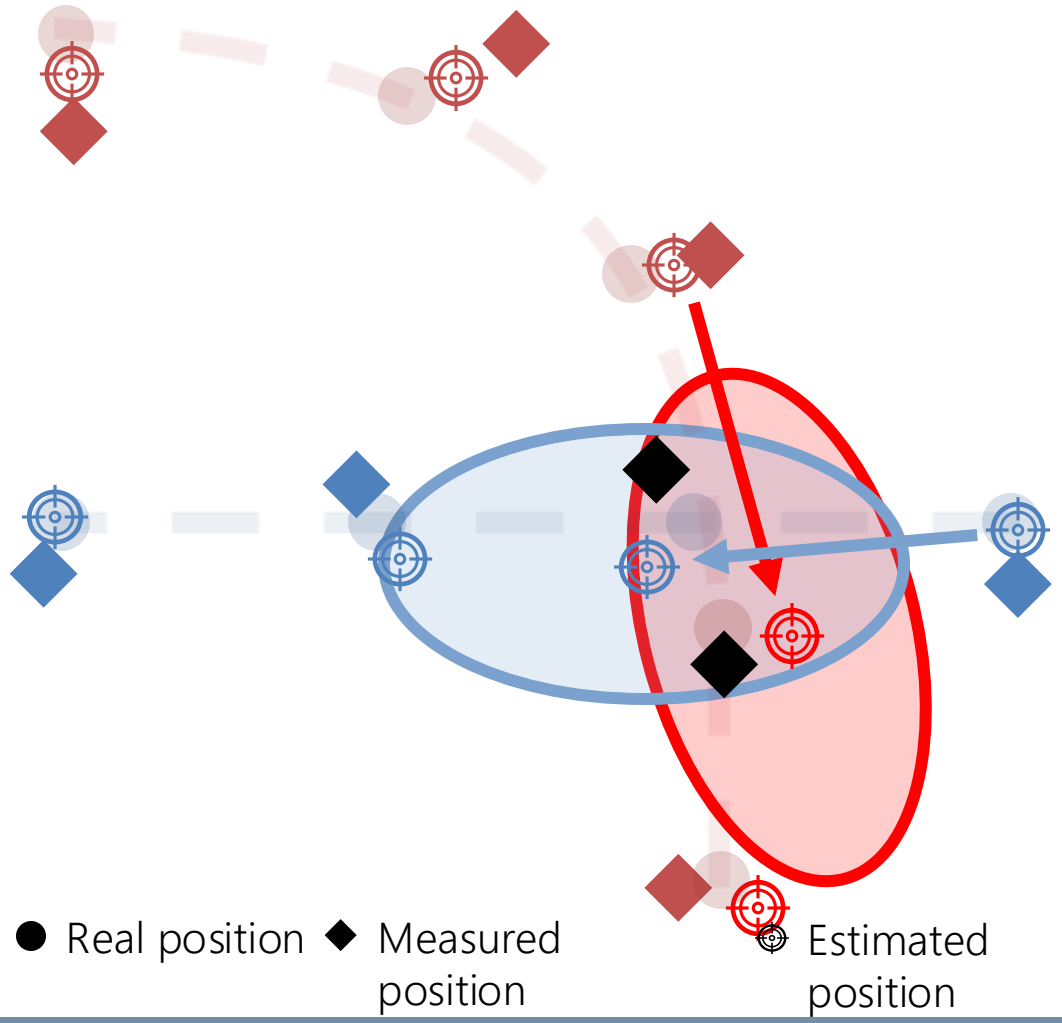


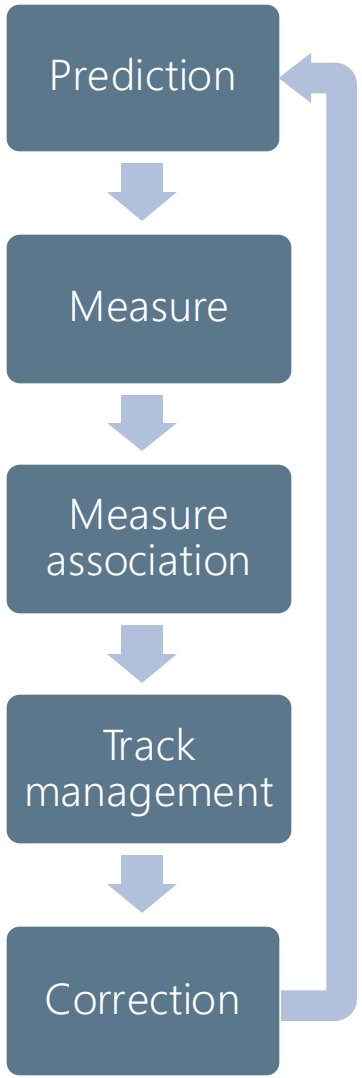
● Real position ◆ Measured position ⊚ Estimated position

Multi-Target Tracking:

1. Unknown number of targets
2. Each target generates at most one measure point

Multi-Target Tracking





Multi-Target Tracking:

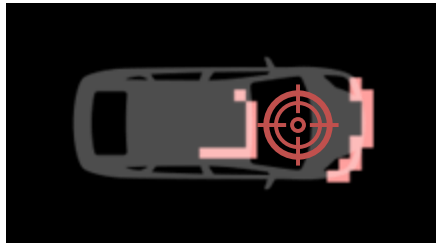
1. Unknown number of targets
2. Each target generates at most one measure point

Extended target tracking

Extended-Target Tracking:

1. Unknown number of targets
2. Each target potentially generates more than one measure point

Measure association problem



Multi-Target Tracking:

1. Unknown number of targets
2. Each target generates at most one measure point



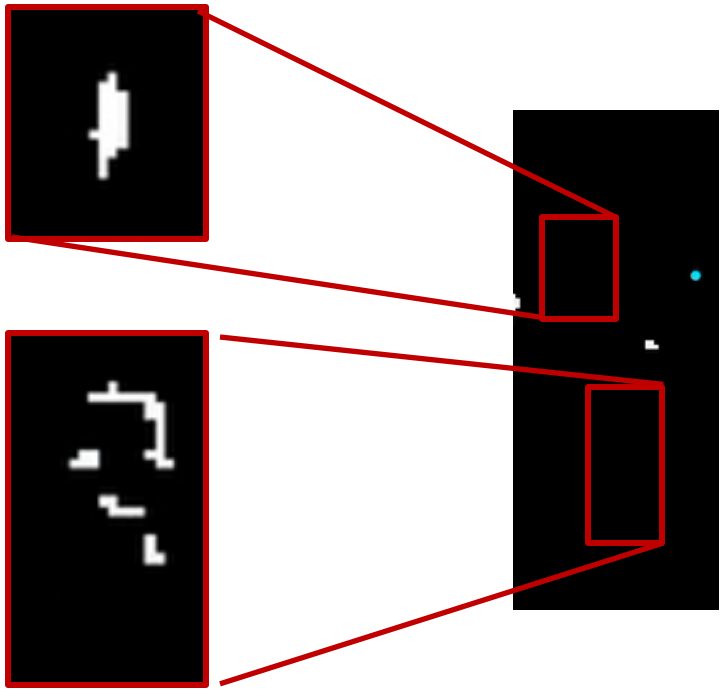
Motorbike

1 target = 1 blob



Car

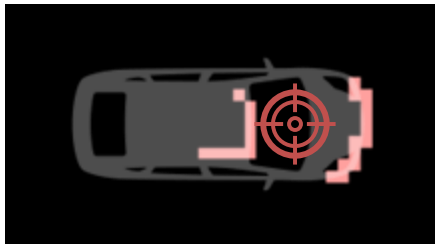
1 target = n blobs



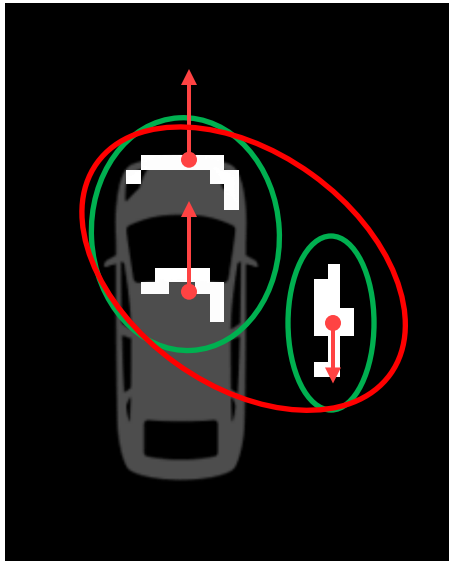
Extended target tracking

Extended targets: generate more than one measure

Measure association problem



Clustering



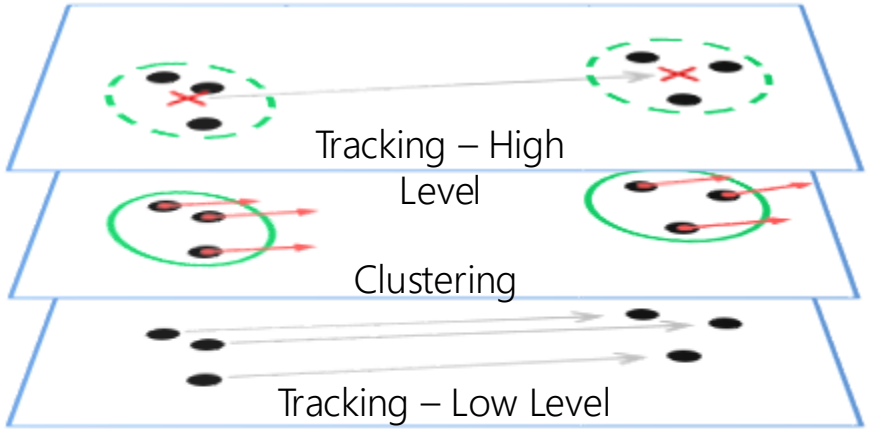
Position-based

Error prone in case of targets close to each other

Position & Speed-based

Improves clustering performance

Two-level algorithm



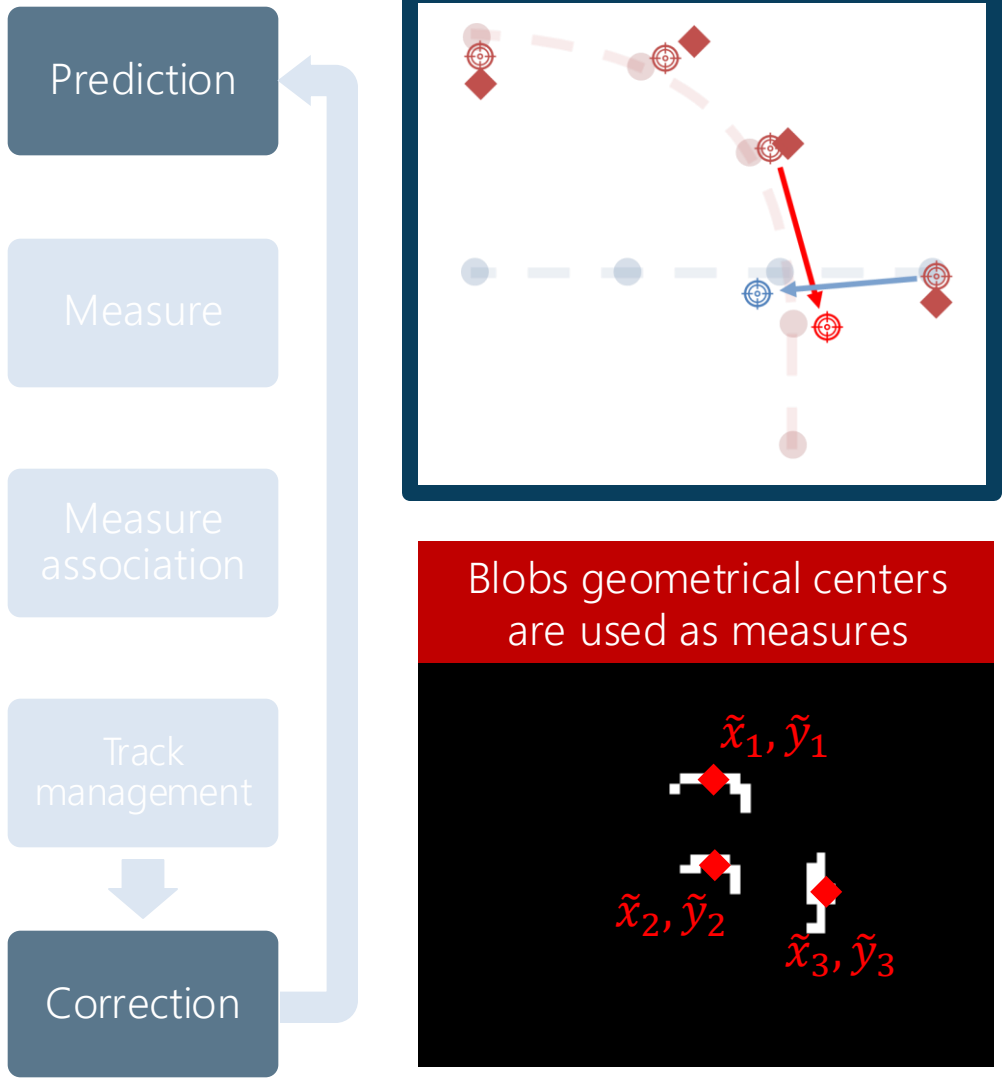
Cluster tracking

Clustering (position & speed-based)

Single components tracking (generate speed measures)

Low Level Tracking

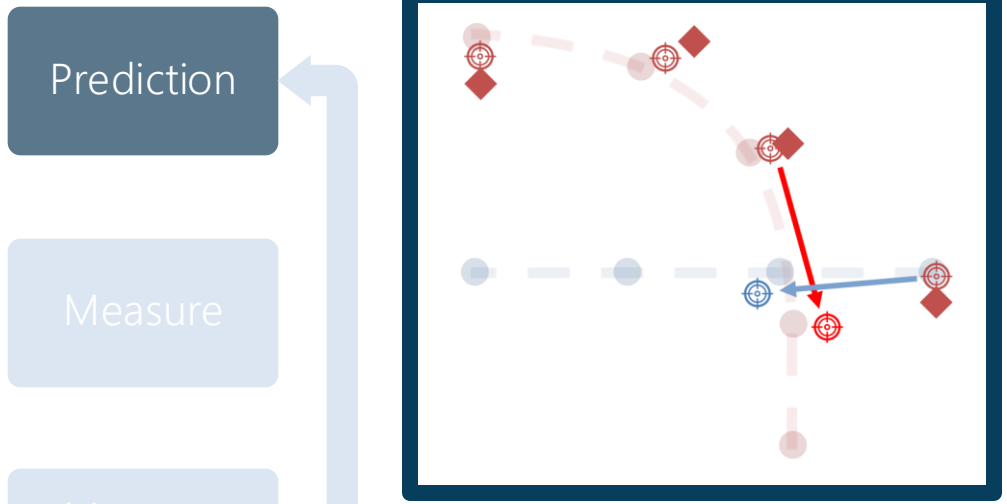
Extended Kalman filter:
coordinated turn motion model



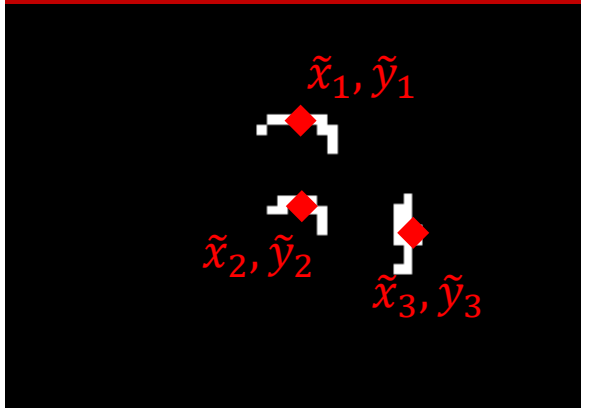
Low Level Tracking

Extended Kalman filter:
coordinated turn motion model

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ v_x \\ v_y \\ \omega \end{bmatrix} \qquad \frac{d|v|}{dt} = 0 \qquad \dot{\omega} = 0$$



Blobs geometrical centers are used as measures



Low Level Tracking

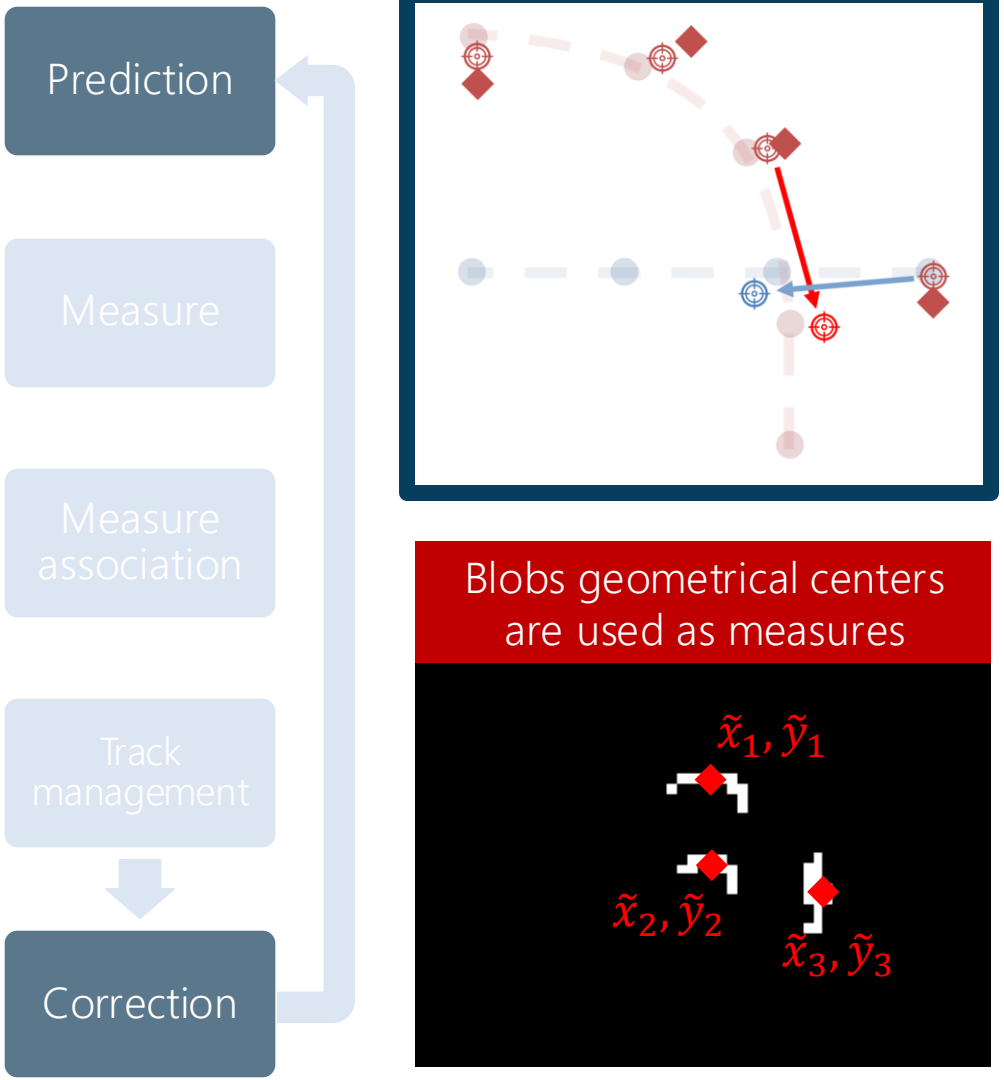
Extended Kalman filter:
coordinated turn motion model

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ v_x \\ v_y \\ \omega \end{bmatrix}$$

$$\frac{d|v|}{dt} = 0$$

$$\dot{\omega} = 0$$

$$\begin{cases} \dot{x} = v_x \\ \dot{y} = v_y \\ \dot{v_x} = -\omega v_y \\ \dot{v_y} = \omega v_x \\ \dot{\omega} = 0 \end{cases}$$



Blobs geometrical centers are used as measures

Low Level Tracking

Extended Kalman filter:
coordinated turn motion model

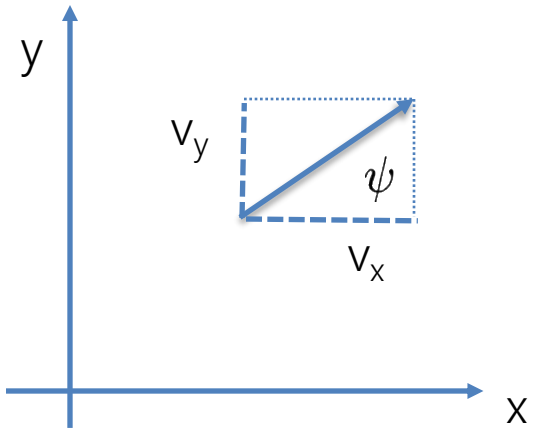
$$\mathbf{x} = \begin{bmatrix} x \\ y \\ v_x \\ v_y \\ \omega \end{bmatrix}$$

$$\frac{d|v|}{dt} = 0$$

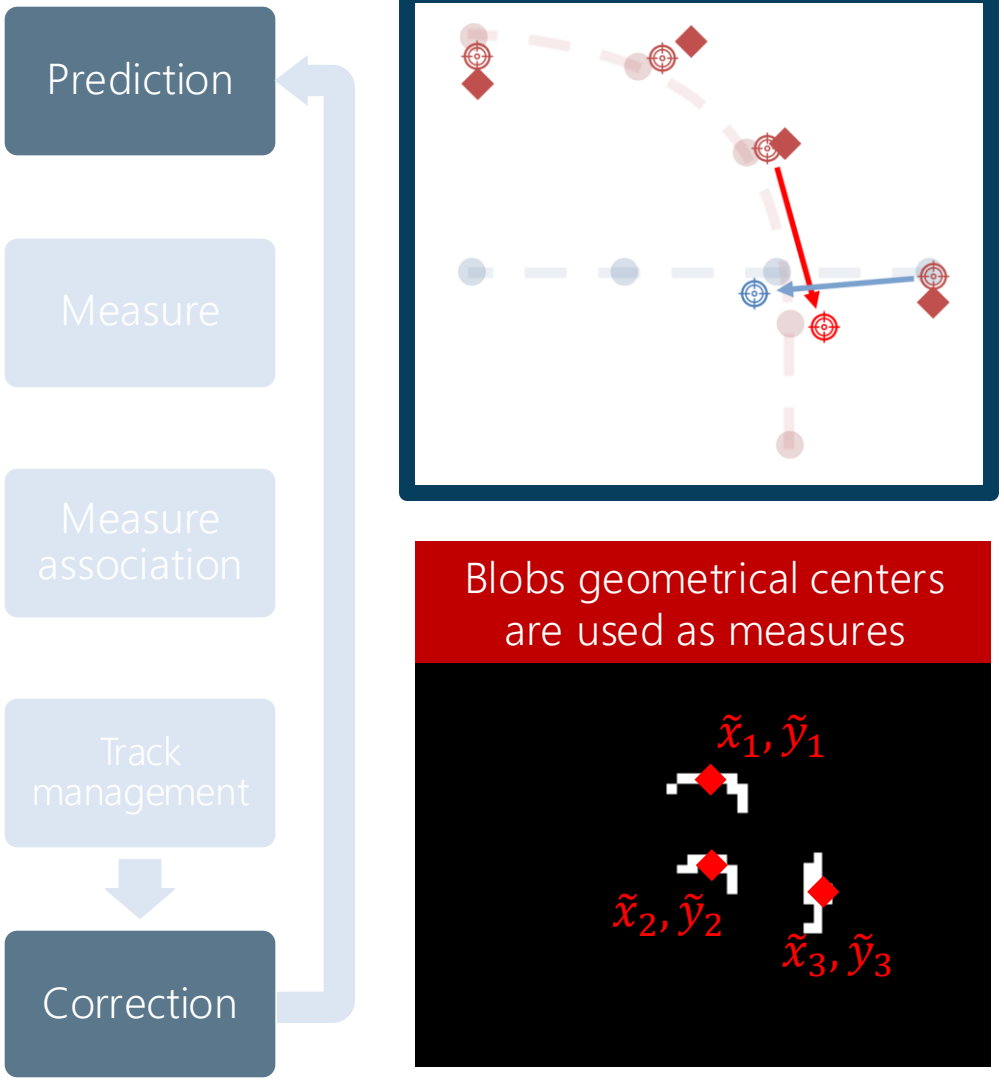
$$\dot{\omega} = 0$$

$$\begin{cases} \dot{x} = v_x \\ \dot{y} = v_y \\ \dot{v_x} = -\omega v_y \\ \dot{v_y} = \omega v_x \\ \dot{\omega} = 0 \end{cases}$$

The model is written in the inertial reference frame



$$v_x = v\cos(\psi)$$
$$v_y = v\sin(\psi)$$



Blobs geometrical centers are used as measures

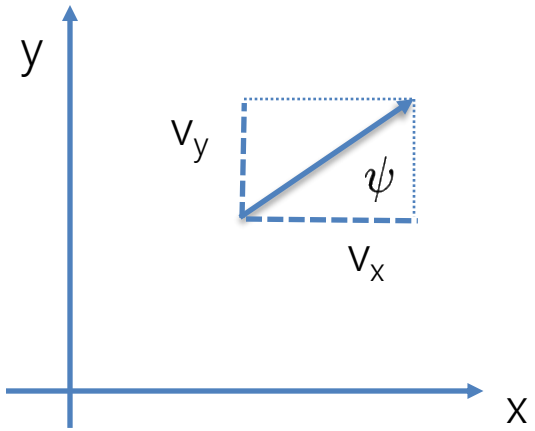
Low Level Tracking

Extended Kalman filter:
coordinated turn motion model

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ v_x \\ v_y \\ \omega \end{bmatrix} \qquad \frac{d|v|}{dt} = 0 \qquad \dot{\omega} = 0$$

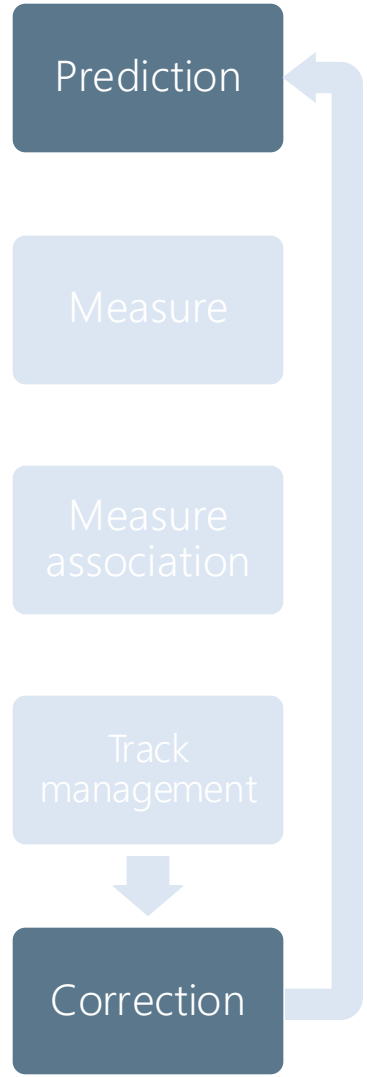
$$\begin{cases} \dot{x} = v_x \\ \dot{y} = v_y \\ \dot{v_x} = -\omega v_y \\ \dot{v_y} = \omega v_x \\ \dot{\omega} = 0 \end{cases}$$

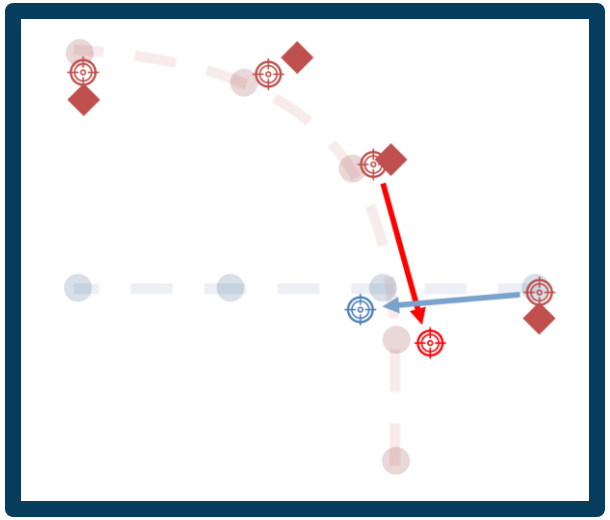
The model is written in the inertial reference frame



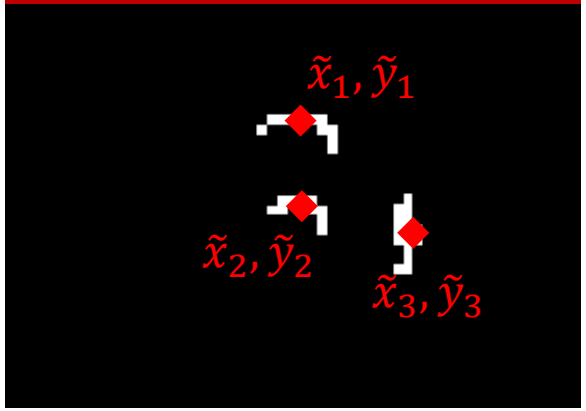
$$\dot{v}_x = -v\sin(\psi)\dot{\psi} + \dot{v}\cos(\psi)$$
$$\dot{v}_y = v\cos(\psi)\dot{\psi} + \dot{v}\sin(\psi)$$

$$v_x = v\cos(\psi)$$
$$v_y = v\sin(\psi)$$





Blobs geometrical centers are used as measures



Low Level Tracking

Extended Kalman filter:
coordinated turn motion model

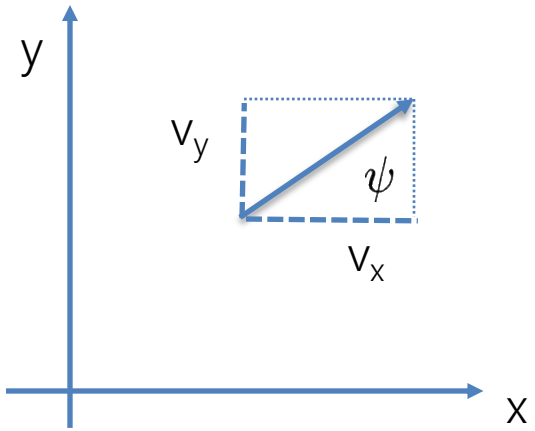
$$\mathbf{x} = \begin{bmatrix} x \\ y \\ v_x \\ v_y \\ \omega \end{bmatrix}$$

$$\frac{d|v|}{dt} = 0$$

$$\dot{\omega} = 0$$

$$\begin{cases} \dot{x} = v_x \\ \dot{y} = v_y \\ \dot{v}_x = -\omega v_y \\ \dot{v}_y = \omega v_x \\ \dot{\omega} = 0 \end{cases}$$

The model is written in the inertial reference frame

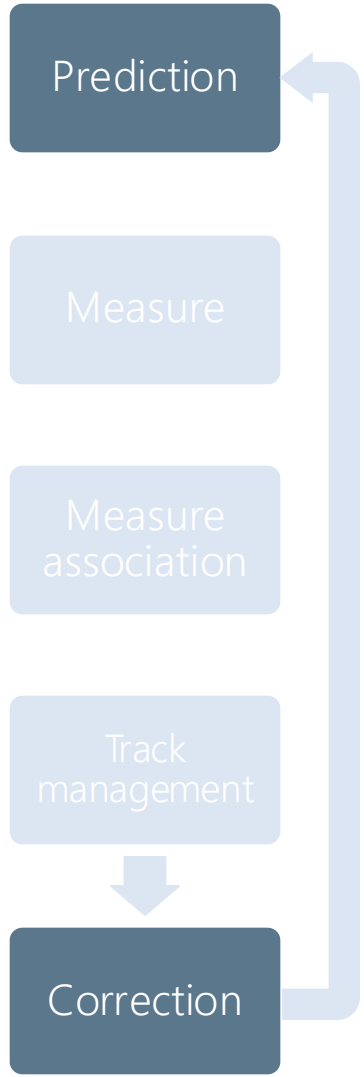


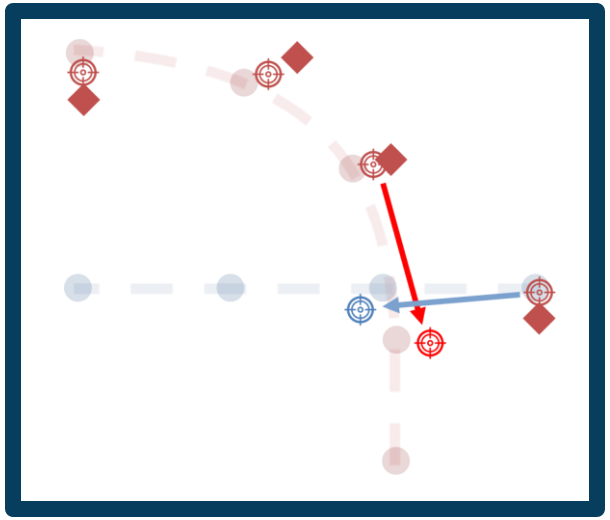
$$\dot{v}_x = -v\sin(\psi)\dot{\psi} + \dot{v}\cos(\psi)$$

$$\dot{v}_y = v\cos(\psi)\dot{\psi} + \dot{v}\sin(\psi)$$

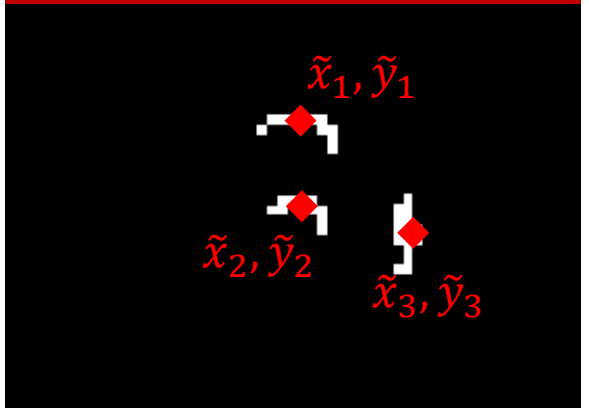
$$\dot{v}_x = -v_y\omega$$

$$\dot{v}_y = v_x\omega$$





Blobs geometrical centers are used as measures



Low Level Tracking

Extended Kalman filter:
coordinated turn motion model

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ v_x \\ v_y \\ \omega \end{bmatrix} \qquad \frac{d|v|}{dt} = 0 \qquad \dot{\omega} = 0$$

$$\begin{cases} \dot{x} = v_x \\ \dot{y} = v_y \\ \dot{v_x} = -\omega v_y \\ \dot{v_y} = \omega v_x \\ \dot{\omega} = 0 \end{cases}$$

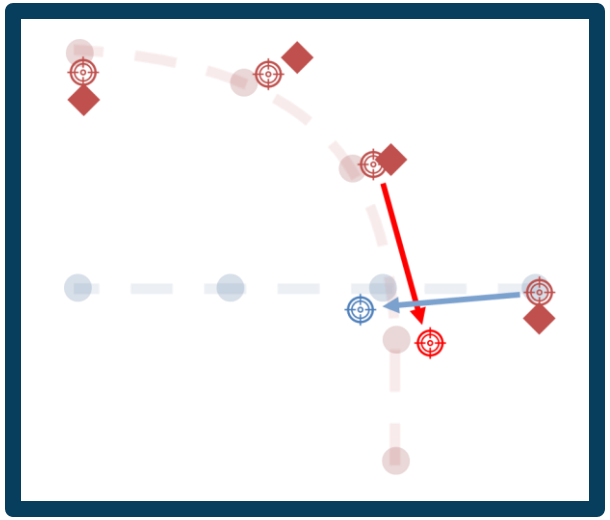
Integrating this system from time k to $\mathrm{k}+1$(over a duration T) yields the matrix

Rows 3 & 4 (Velocity):

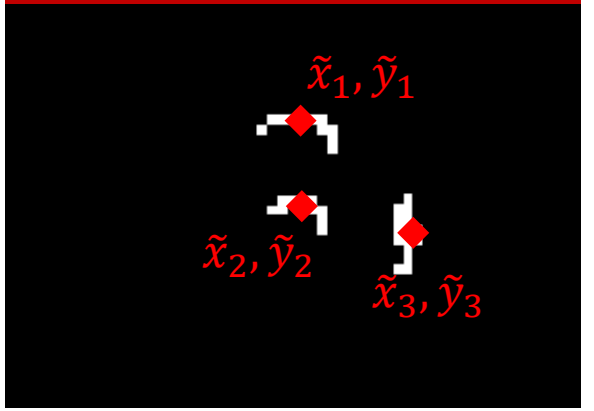
Integrating $\dot{v}_y = -v_x\omega$ and $\dot{v}_x = -v_y\omega$ is a rotation:

$$\begin{bmatrix} v_{x,k+1} \\ v_{y,k+1} \end{bmatrix} = \begin{bmatrix} \cos(\omega_k T) & -\sin(\omega_k T) \\ \sin(\omega_k T) & \cos(\omega_k T) \end{bmatrix} \begin{bmatrix} v_{x,k} \\ v_{y,k} \end{bmatrix}$$





Blobs geometrical centers are used as measures



Extended Kalman filter:
coordinated turn motion model

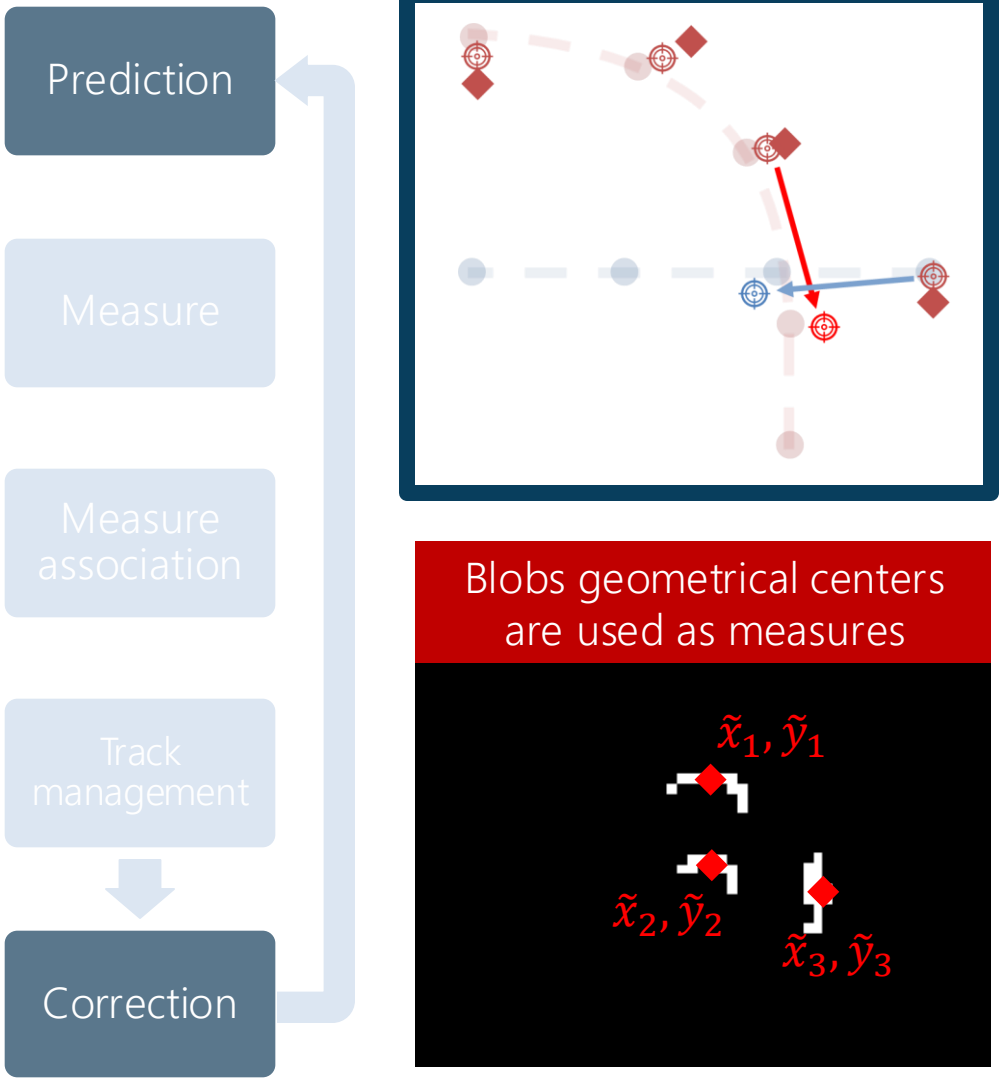
Rows 1 & 2 (Position): These are found by integrating the velocity solution (if $\omega \neq 0$):

$$x_{k+1} = x_k + \int_0^T v_x(t)dt$$

considering

$$v_x(t) = \cos(\omega_k t)v_{x,k} - \sin(\omega_k t)v_{y,k}$$

$$x_{k+1} = x_k + \frac{\sin(\omega_k T)}{\omega_k}v_{x,k} + \frac{\cos(\omega_k T) - 1}{\omega_k}v_{y,k}$$



Extended Kalman filter:
coordinated turn motion model

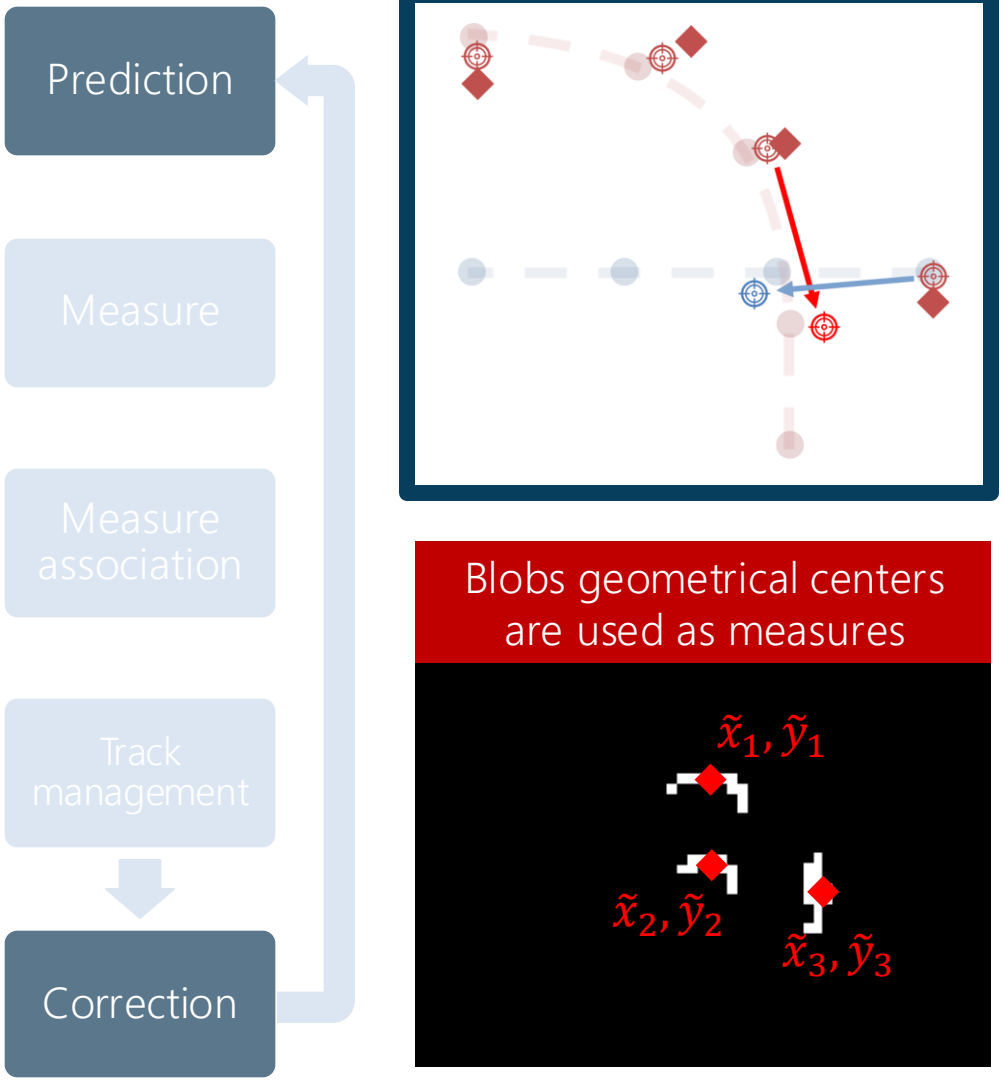
Rows 1 & 2 (Position): These are found by integrating the velocity solution (if $\omega \neq 0$):

$$y_{k+1} = y_k + \int_0^T v_y(t)dt$$

considering

$$v_y(t) = \sin(\omega_k t)v_{x,k} + \cos(\omega_k t)v_{y,k}$$

$$y_{k+1} = y_k + \frac{1-\cos(\omega_k T)}{\omega_k}v_{x,k} + \frac{\sin(\omega_k T)}{\omega_k}v_{y,k}$$



Low Level Tracking

Extended Kalman filter:
coordinated turn motion model

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ v_x \\ v_y \\ \omega \end{bmatrix}$$

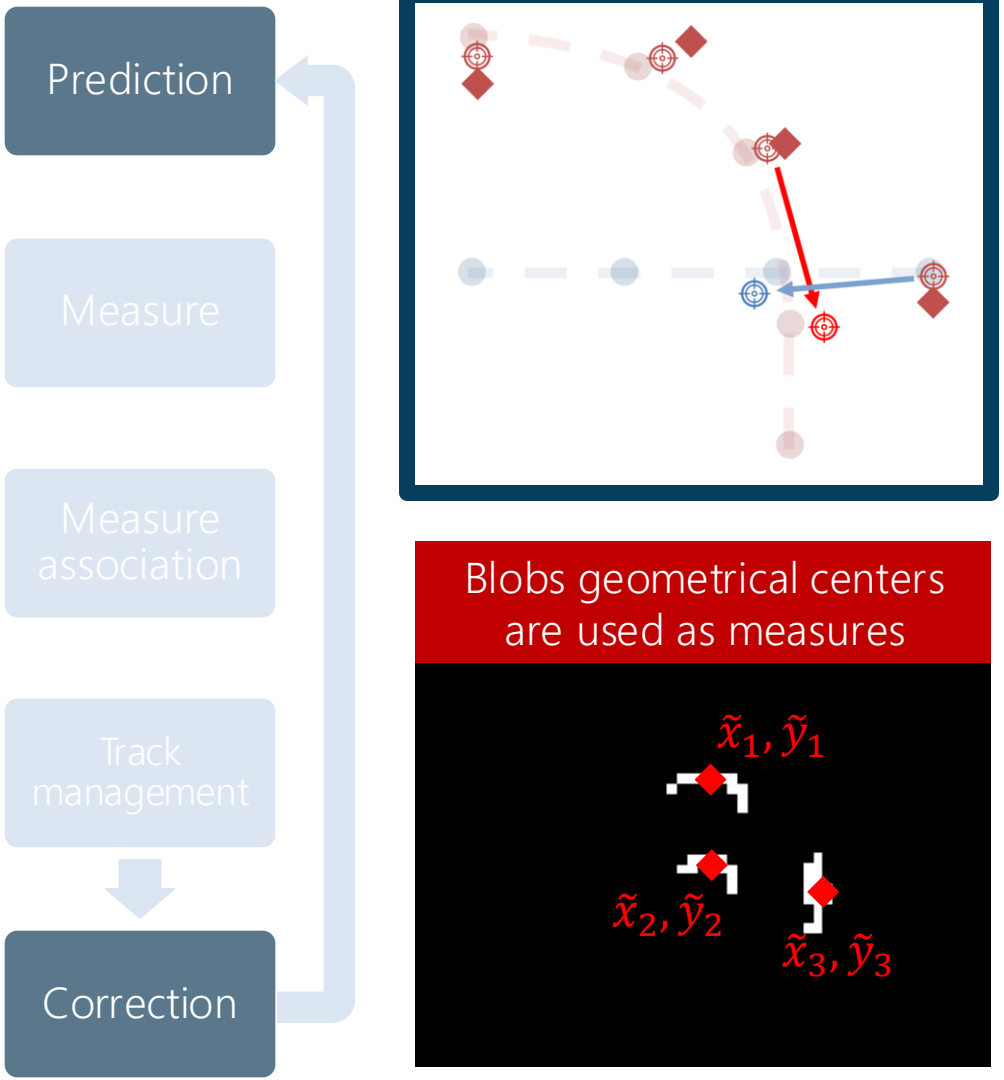
$$\frac{d|v|}{dt} = 0$$

$$\dot{\omega} = 0$$

$$\begin{cases} \dot{x} = v_x \\ \dot{y} = v_y \\ \dot{v_x} = -\omega v_y \\ \dot{v_y} = \omega v_x \\ \dot{\omega} = 0 \end{cases}$$

Row 5 (Turn Rate):

$$\omega_{k+1} = \omega_k$$



Blobs geometrical centers are used as measures

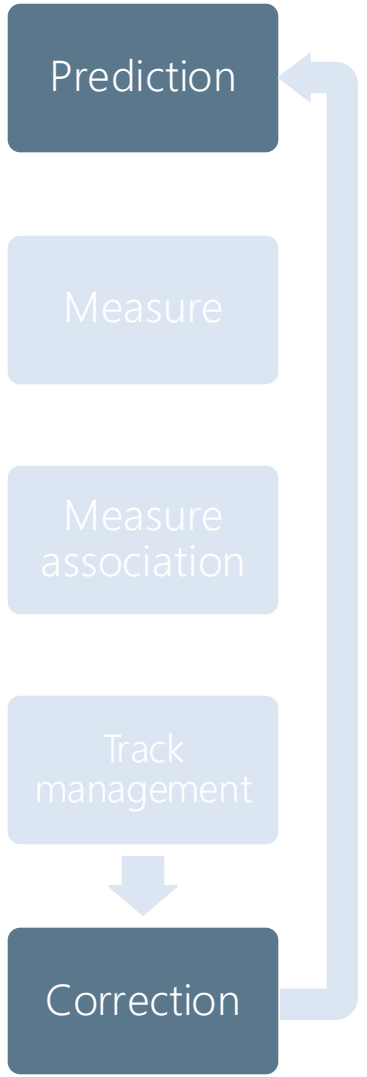
Extended Kalman filter:
coordinated turn motion model

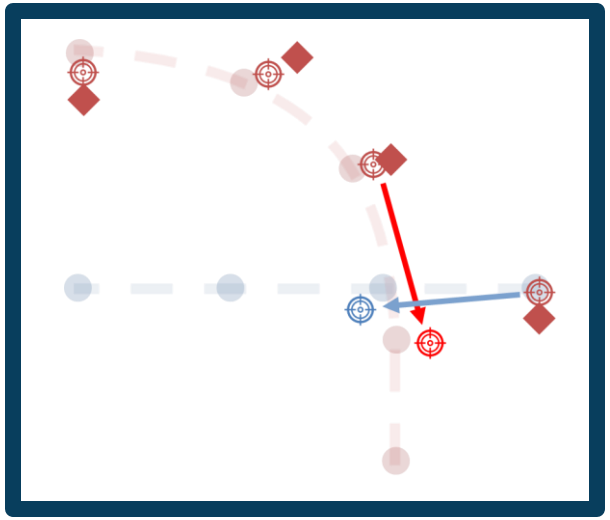
Derivation of the 5x3 Noise Matrix

This matrix, G_k, represents the **stochastic** (or input) part of the motion. It models how the state is affected by a small, unmodeled acceleration,

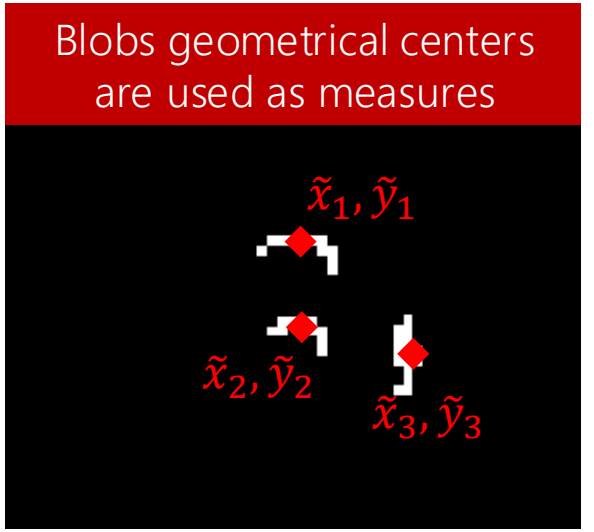
$$\mathbf{w}_k = [a_x, a_y, \alpha]^T$$

This part is derived by assuming a *different*, simpler model: that the noise w_k is a **constant** acceleration applied over the interval T. This is a standard Constant Acceleration model.





Blobs geometrical centers are used as measures



Low Level Tracking

Extended Kalman filter:
coordinated turn motion model

Derivation of the 5x3 Noise Matrix

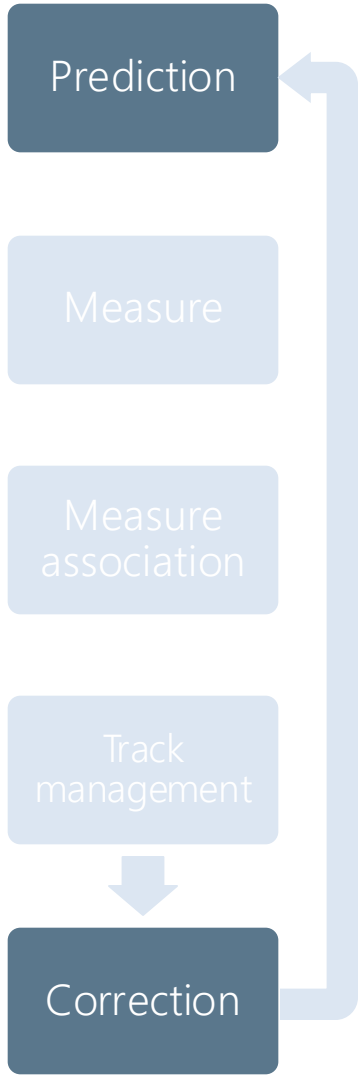
Rows 1 & 3 (x-motion):
If we have a constant acceleration a_x, the continuous model is

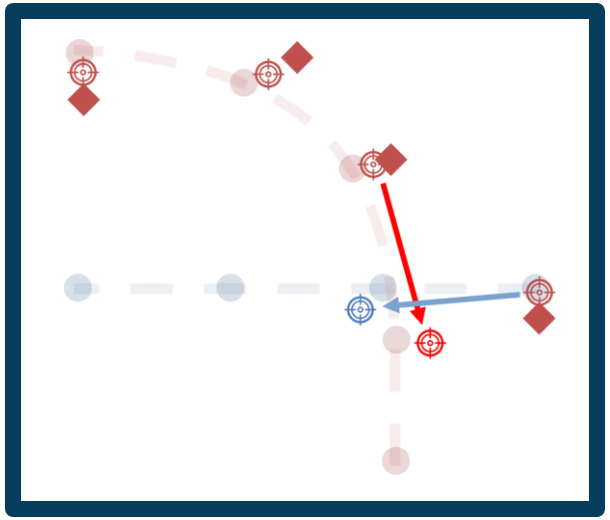
$$\dot{x} = v_x$$
$$\dot{v}_x = a_x$$

Integrating $\dot{v}_x = a_x$ gives $v_{k+1} = v_k + a_x T$

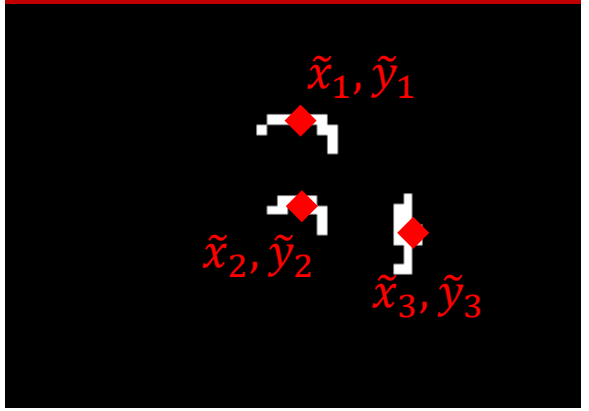
Integrating $\dot{x} = v_x$ gives $x_{k+1} = x_k + v_k T + \frac{1}{2} a_x T^2$

Rows 2 & 4 (y-motion):
The same logic applies independently to y and v_y with acceleration a_y





Blobs geometrical centers are used as measures



Low Level Tracking

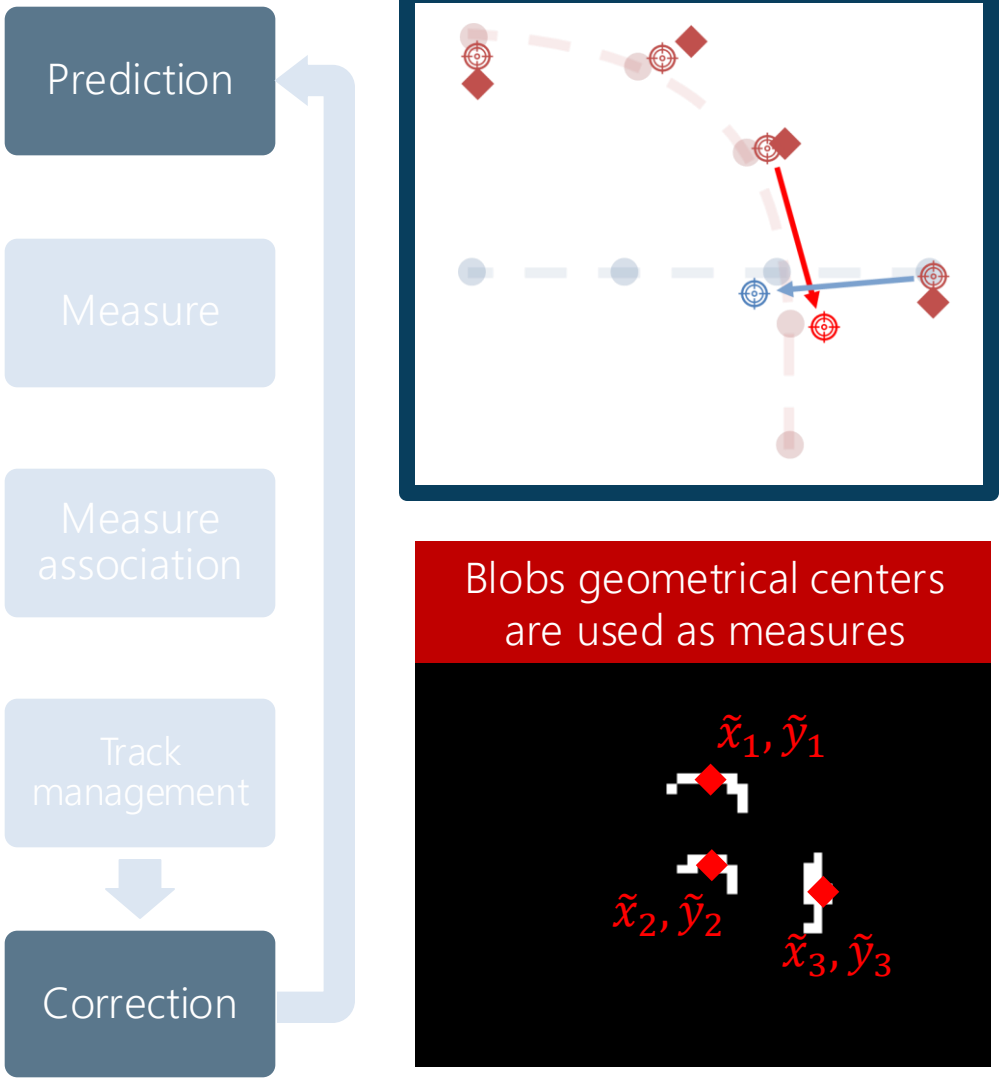
Extended Kalman filter:
coordinated turn motion model

Derivation of the 5x3 Noise Matrix

Row 5 (Turn Rate):

The same logic applies to ω with turn acceleration α yielding

$$\omega_{k+1} = \omega_k + \alpha T$$



Low Level Tracking

Extended Kalman filter:
coordinated turn motion model

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ v_x \\ v_y \\ \omega \end{bmatrix} \qquad \frac{d|v|}{dt} = 0 \qquad \dot{\omega} = 0$$

$$\begin{cases} \dot{x} = v_x \\ \dot{y} = v_y \\ \dot{v}_x = -\omega v_y \\ \dot{v}_y = \omega v_x \\ \dot{\omega} = 0 \end{cases}$$

$$\boldsymbol{x}_{k+1} = \begin{bmatrix} 1 & \frac{\sin(\omega_k T)}{\omega_k} & 0 & -\frac{1-\cos(\omega_k T)}{\omega_k} & 0 \\ 0 & \cos(\omega_k T) & 0 & -\sin(\omega_k T) & 0 \\ 0 & \frac{1-\cos(\omega_k T)}{\omega_k} & 1 & \frac{\sin(\omega_k T)}{\omega_k} & 0 \\ 0 & \sin(\omega_k T) & 0 & \cos(\omega_k T) & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \boldsymbol{x}_k + \begin{bmatrix} \frac{T^2}{2} & 0 & 0 \\ T & 0 & 0 \\ 0 & \frac{T^2}{2} & 0 \\ 0 & T & 0 \\ 0 & 0 & T \end{bmatrix} \boldsymbol{w}_k, \quad \boldsymbol{w}_k = \begin{bmatrix} a_x \\ a_y \\ \alpha \end{bmatrix} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$$

$$\boldsymbol{y}_k = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} \boldsymbol{x}_k + \boldsymbol{v}_k, \quad \boldsymbol{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$$

Only position is measured

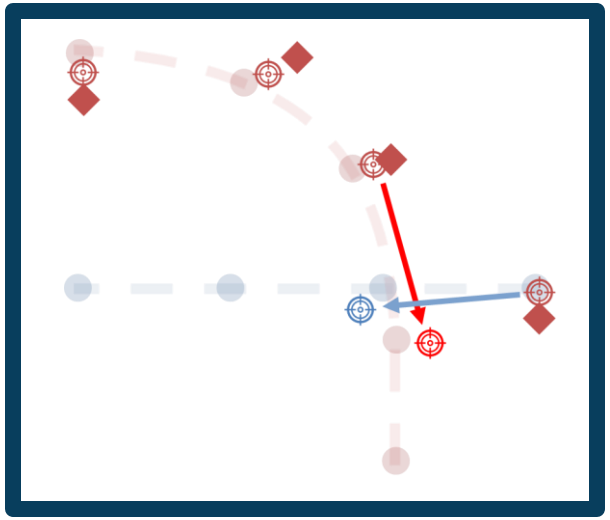
Prediction

Measure

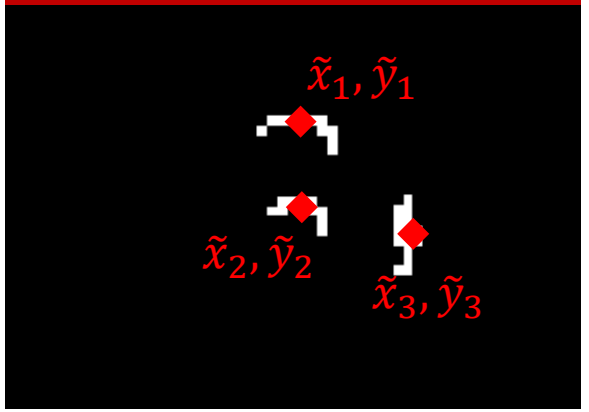
Measure association

Track management

Correction



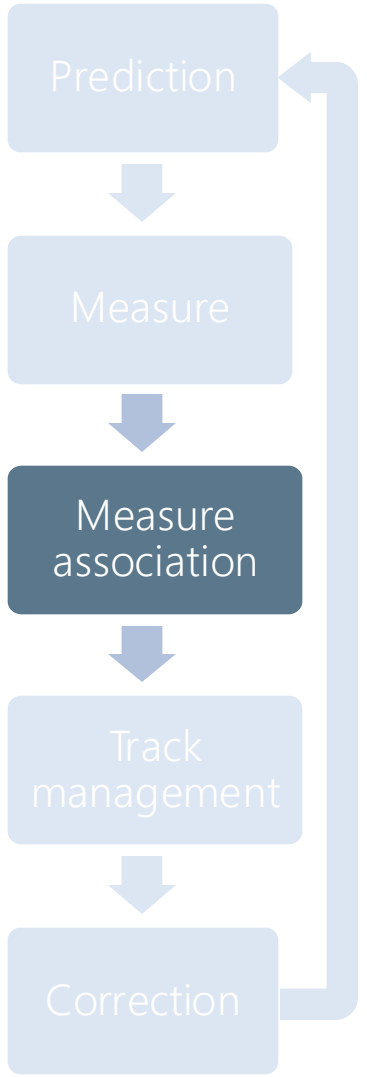
Blobs geometrical centers are used as measures

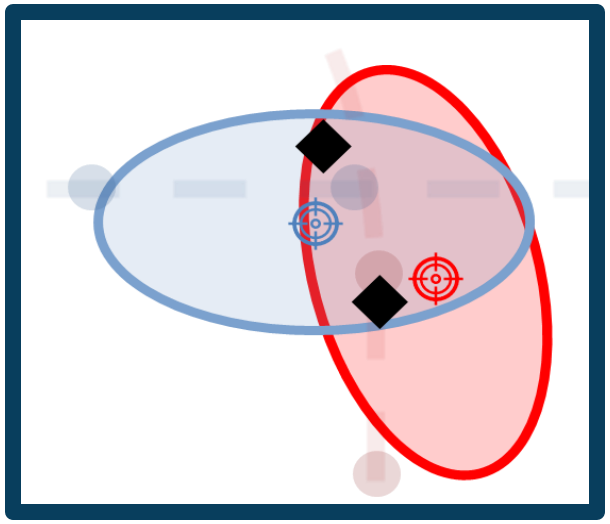


Low Level Tracking – Measure Association

Two step approach:

- Gating
- Association





Low Level Tracking – Measure Association

Two step approach:

- **Gating**
- Association

For each track we determine the compatible measurement

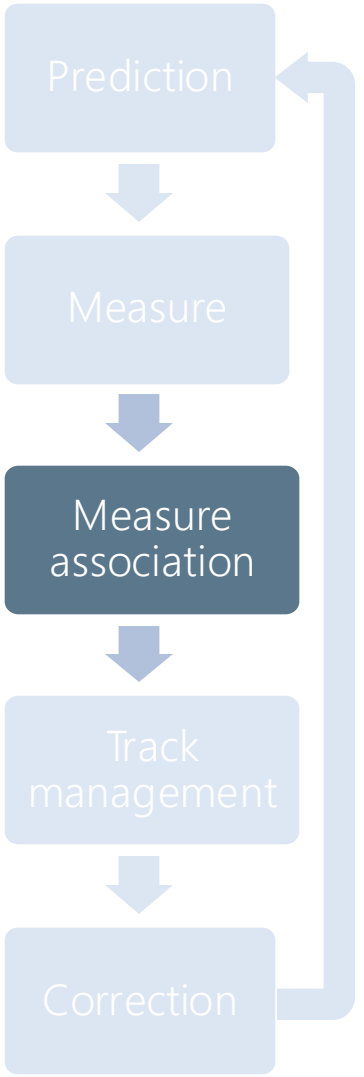
$$D_{n,m} = \sqrt{(\mathbf{y}_m - \hat{\mathbf{y}}_n)^T S^{-1} (\mathbf{y}_m - \hat{\mathbf{y}}_n)}$$

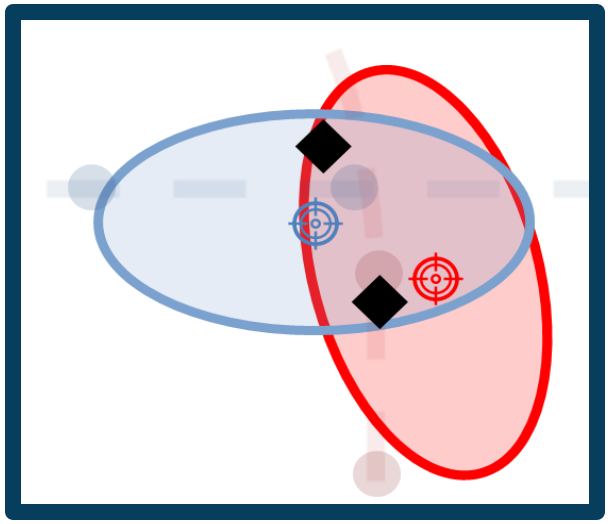
Mahalanobis Distance

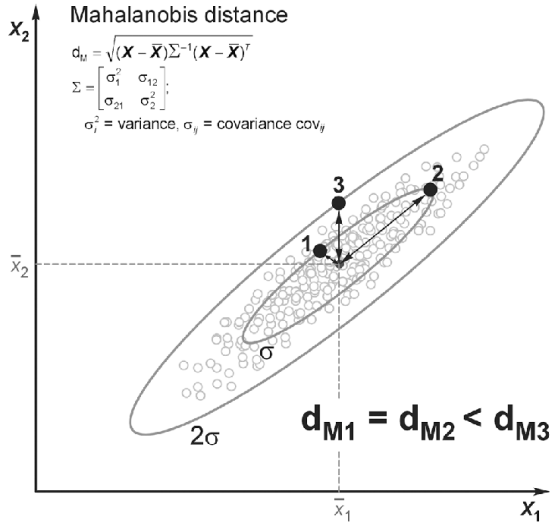
$$S = HP'H^\top + R.$$

$$d_{n,m} = \begin{cases} D_{n,m} & if\ D_{n,m} \leq \tau_G \\ \infty & otherwise \end{cases}$$

τ_G = *Gating Threshold*







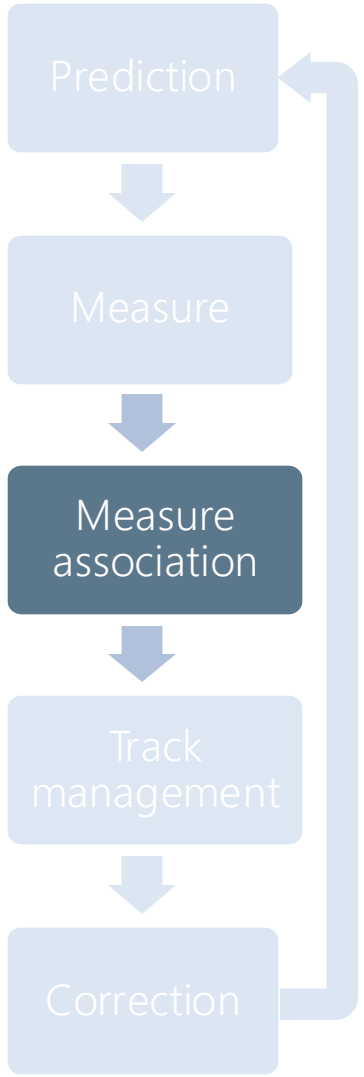
Low Level Tracking – Measure Association

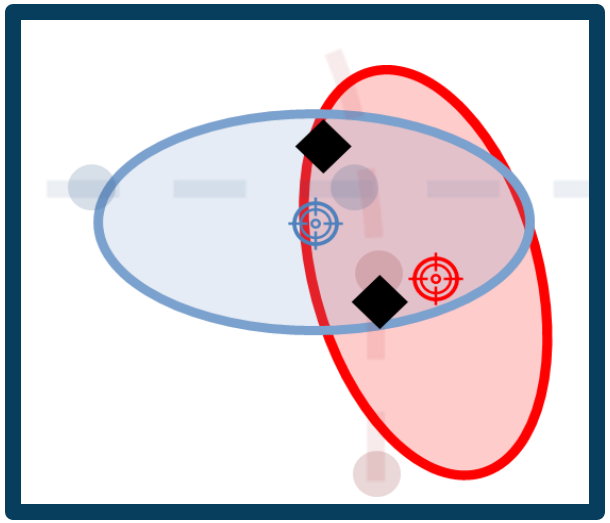
Two step approach:

- Gating
- **Association**

More than one measure can satisfy (the gating condition for the same track, and conversely the same measure can be acceptable for more than one track.

Data association done with Global Nearest Neighbor (GNN)

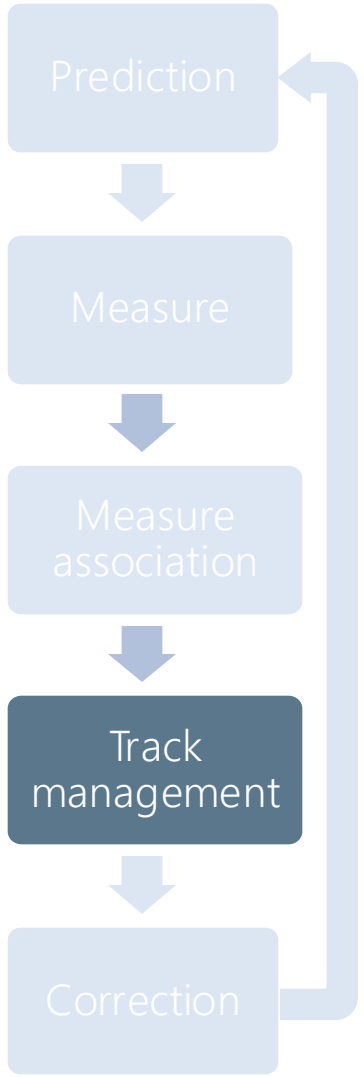


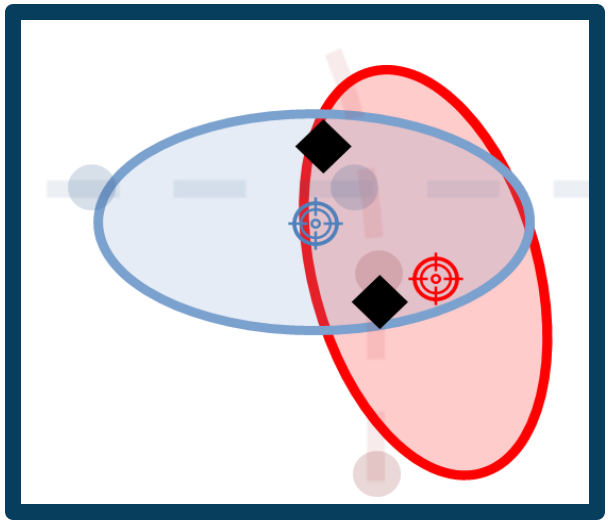


Low Level Tracking

Track management: M/N logic

- A new track is initialized for each unassociated measure
- M measure in the first N instants: track confirmed
- Less then M measure in the last N instants: track eliminated





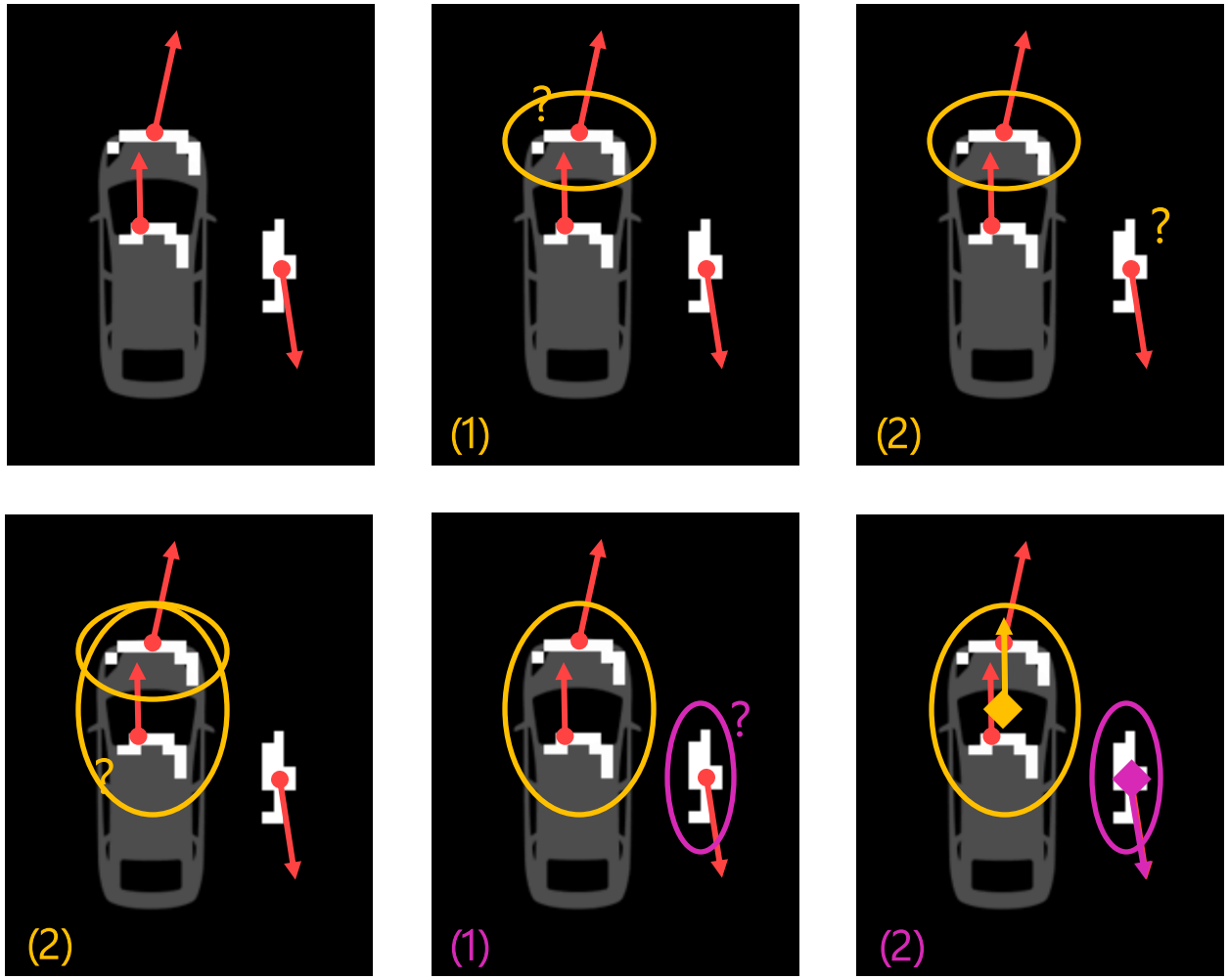
Low Level Tracking - results



Measures are considered only within the road and sidewalk boundaries

Tentative tracks Confirmed tracks Background YAPE

Double-Threshold Based Clustering



Position threshold

$$\delta_d(p, C) = \min_{q \in C} \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2}$$

Speed threshold

$$\delta_v(p, C) = \max_{q \in C} \sqrt{\left(v_{x_p} - v_{x_q}\right)^2 + \left(v_{y_p} - v_{y_q}\right)^2}$$

The Algorithm:

1. Randomly select a track as the seed of the first cluster C_1
2. For every unclustered track p compute $\delta_d(p, C_1)$ and $\delta_v(p, C_1)$; p is included in C_1 if $\delta_d(p, C_1)$ and $\delta_v(p, C_1)$ are below two prescribed thresholds
3. If there are still unclustered track, randomly select one to be the seed of the next cluster, then repeat step 2 for the new cluster.

High Level Tracking

Extended Kalman filter

Coordinated turn motion model

Prediction

Cluster tracks average position and speed

Measure

Gating (Mahalanobis)

Global Nearest Neighbor

Measure association

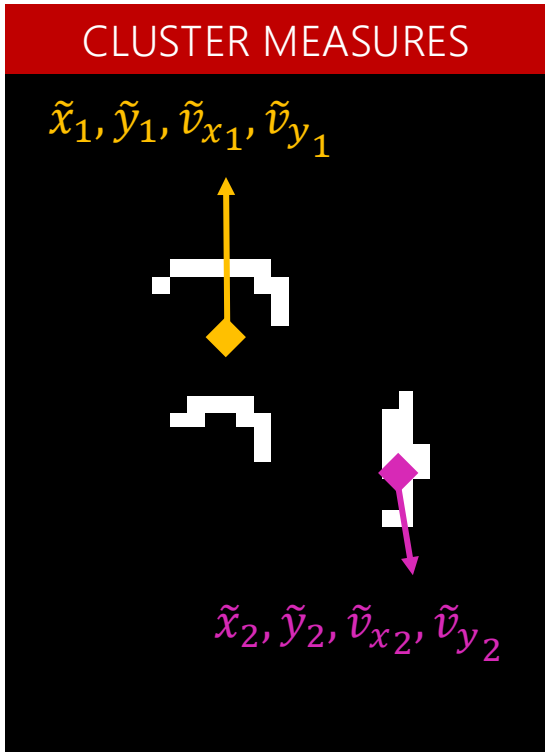
New cluster per each unassociated measure

M/N management

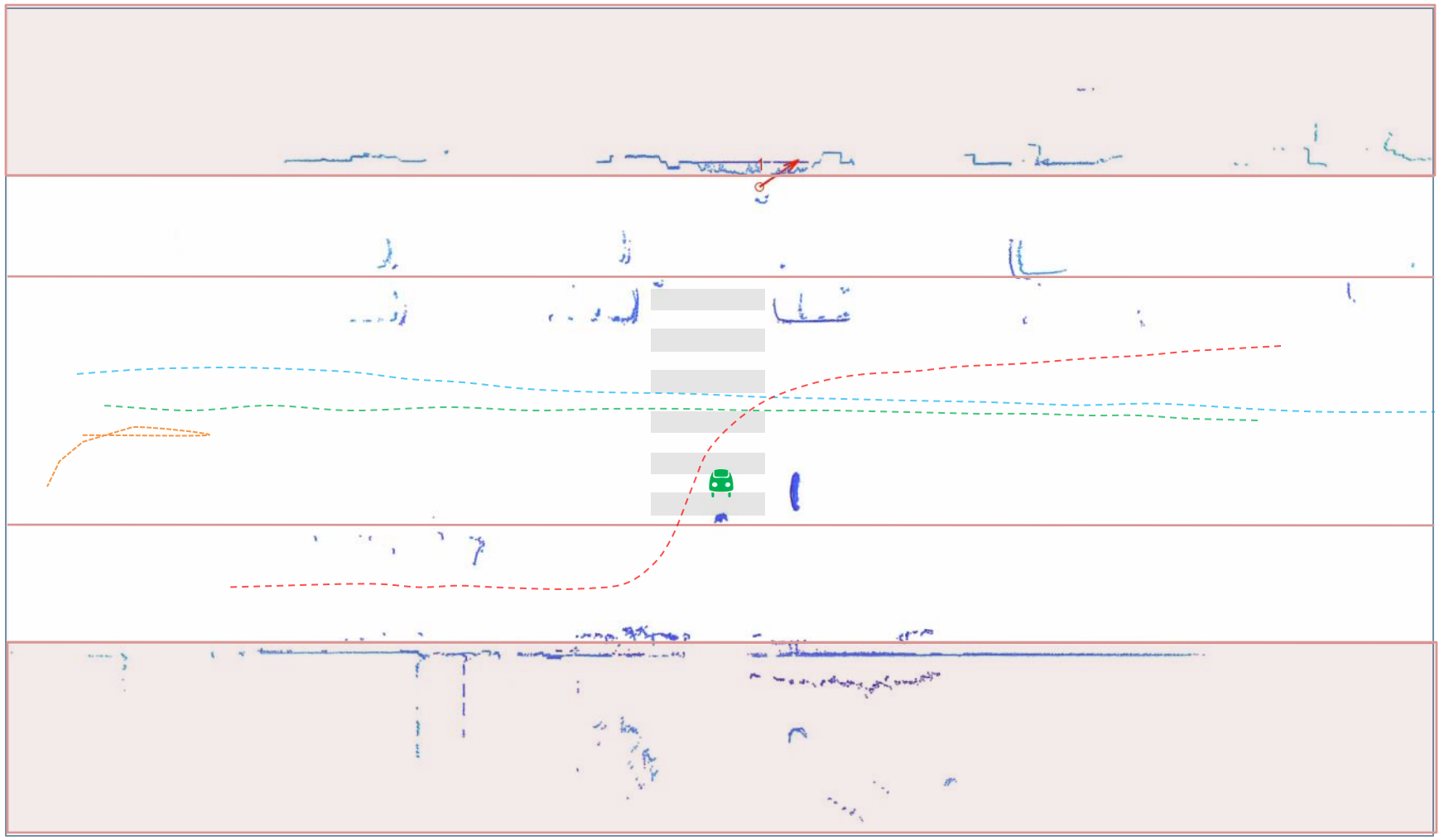
Track management

EKF averaging tracks position and speed

Correction



Final Results



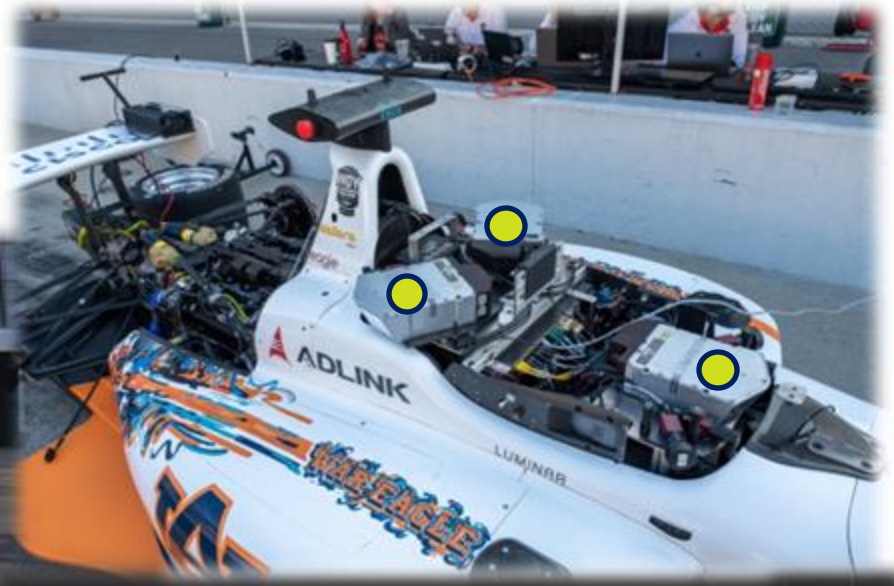
User selected ground truth

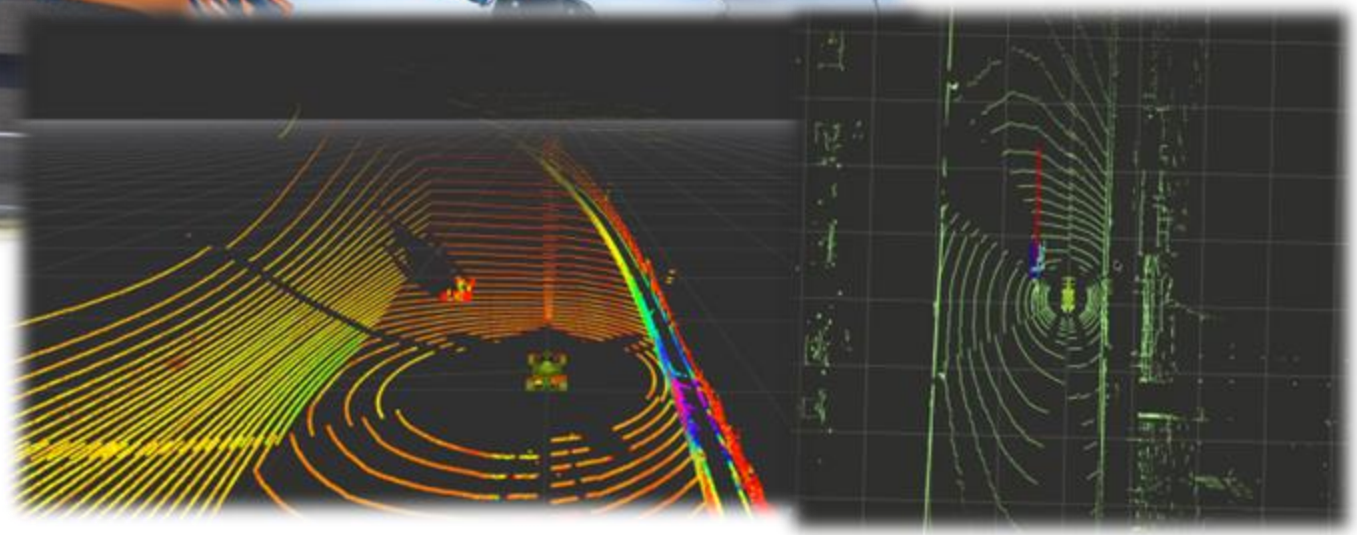
Estimated trajectory

YAPE

Perception and Target tracking for Autonomous Racing



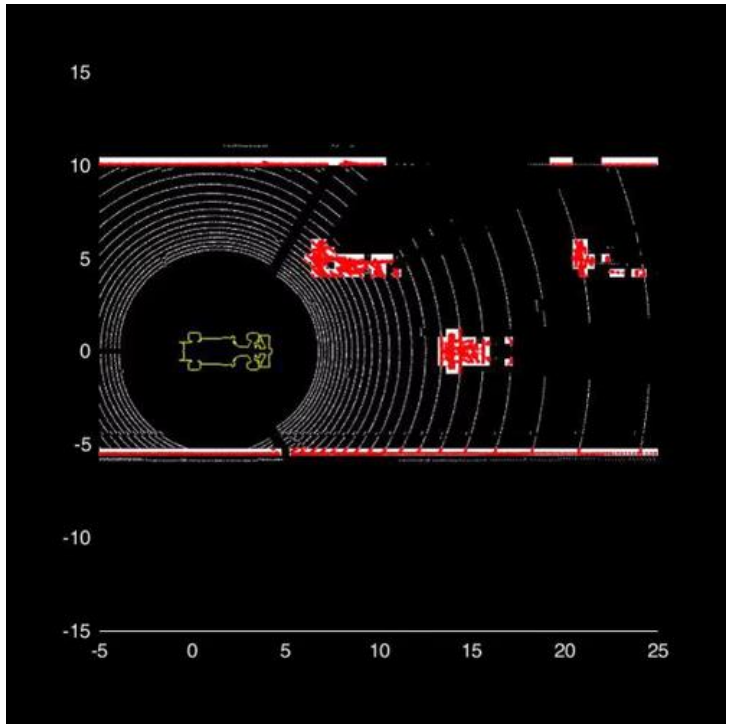


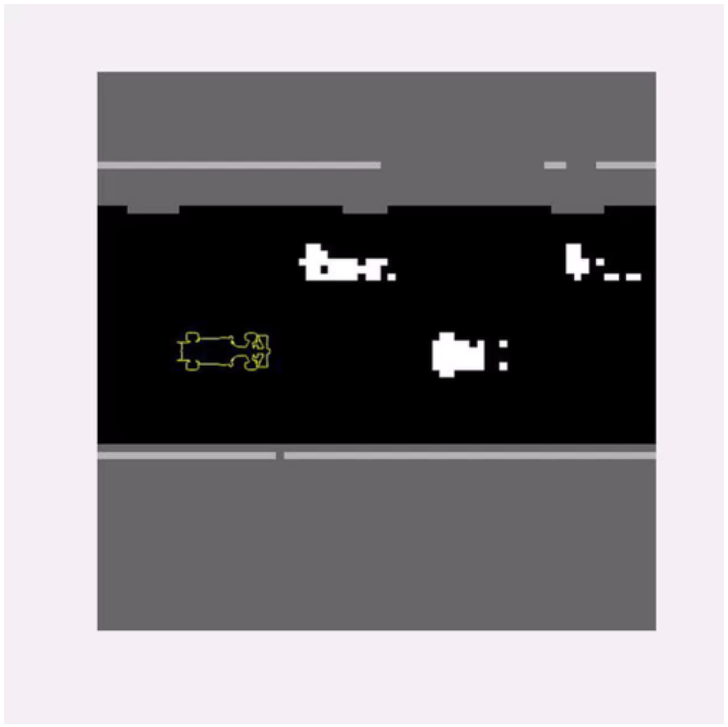


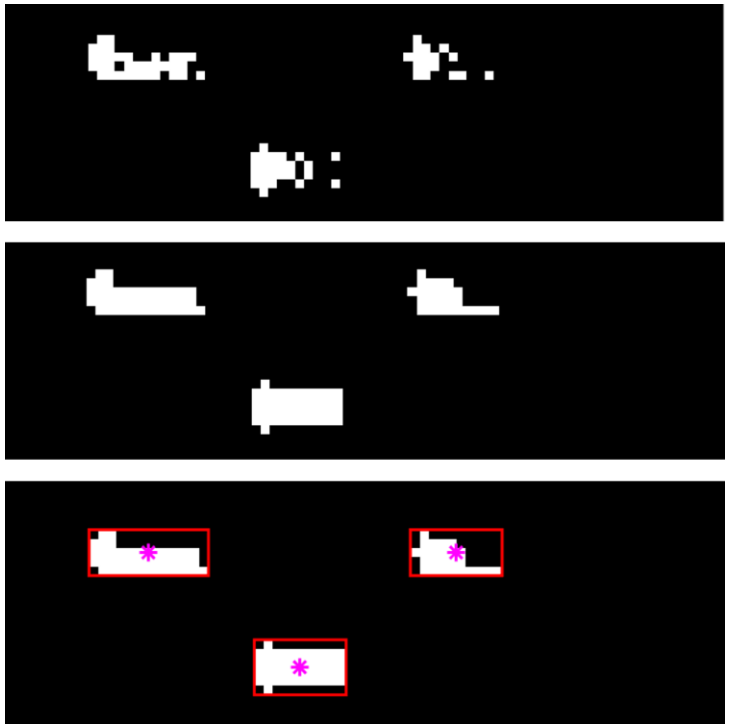
- Luminar H3 vertically scanning LiDARs

- 32 vertical scanning layers (@20 Hz)
- 120° Horizontal FOV
- 0-30° Vertical FOV

Pointcloud Processing







1) Ground Removal

Grid-based looking at maximum height difference in every cell

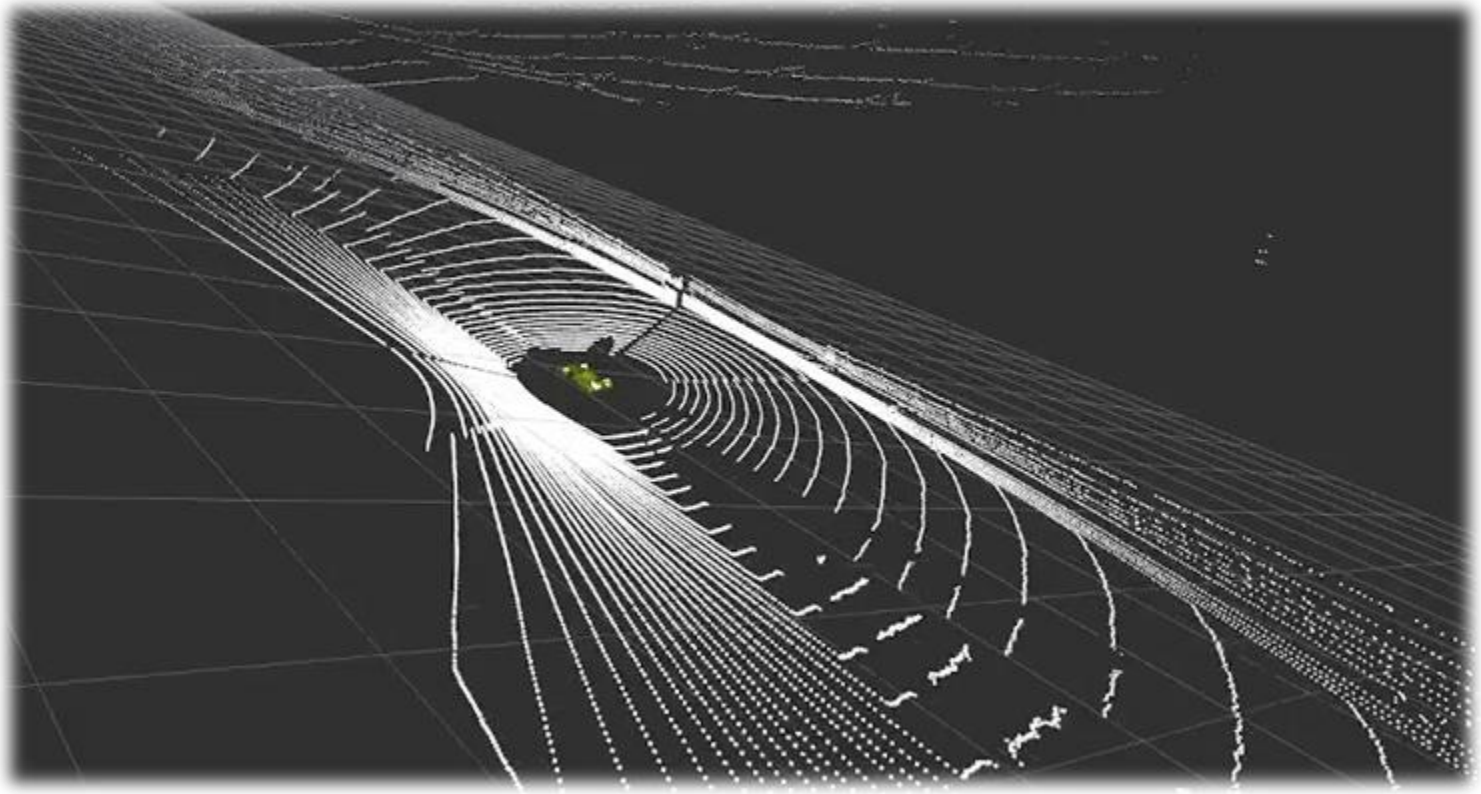
2) Bounds Removal

Map-based using current localization estimate

3) Measures Extraction

Morphological closing to connect pixels. Area filtering and centroid extraction.

Autonomous Racing Challenges



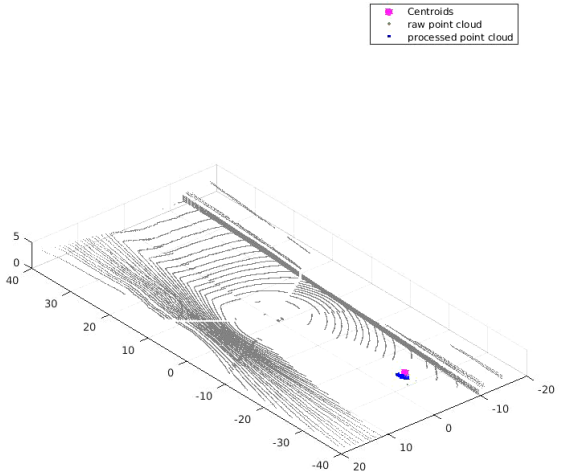
Racing Challenges

1. Accurate bound removal [wall detection]
2. Shadow Management [rear wing blind region]
3. Noise filtering [Track sand detection]

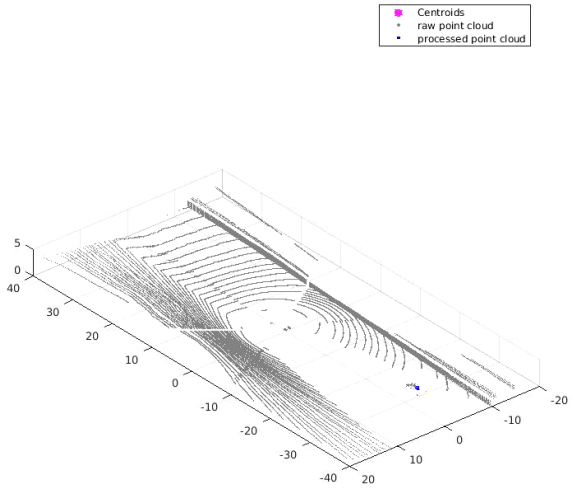
Track sand detection

- LiDAR sensor returns a measurement of intensity (reflecitivity) for every point
- Points belonging to the «sand tail» have very low intensity (reflectivity)

Discard low reflectivity points









Target tracking in the race



Prediction

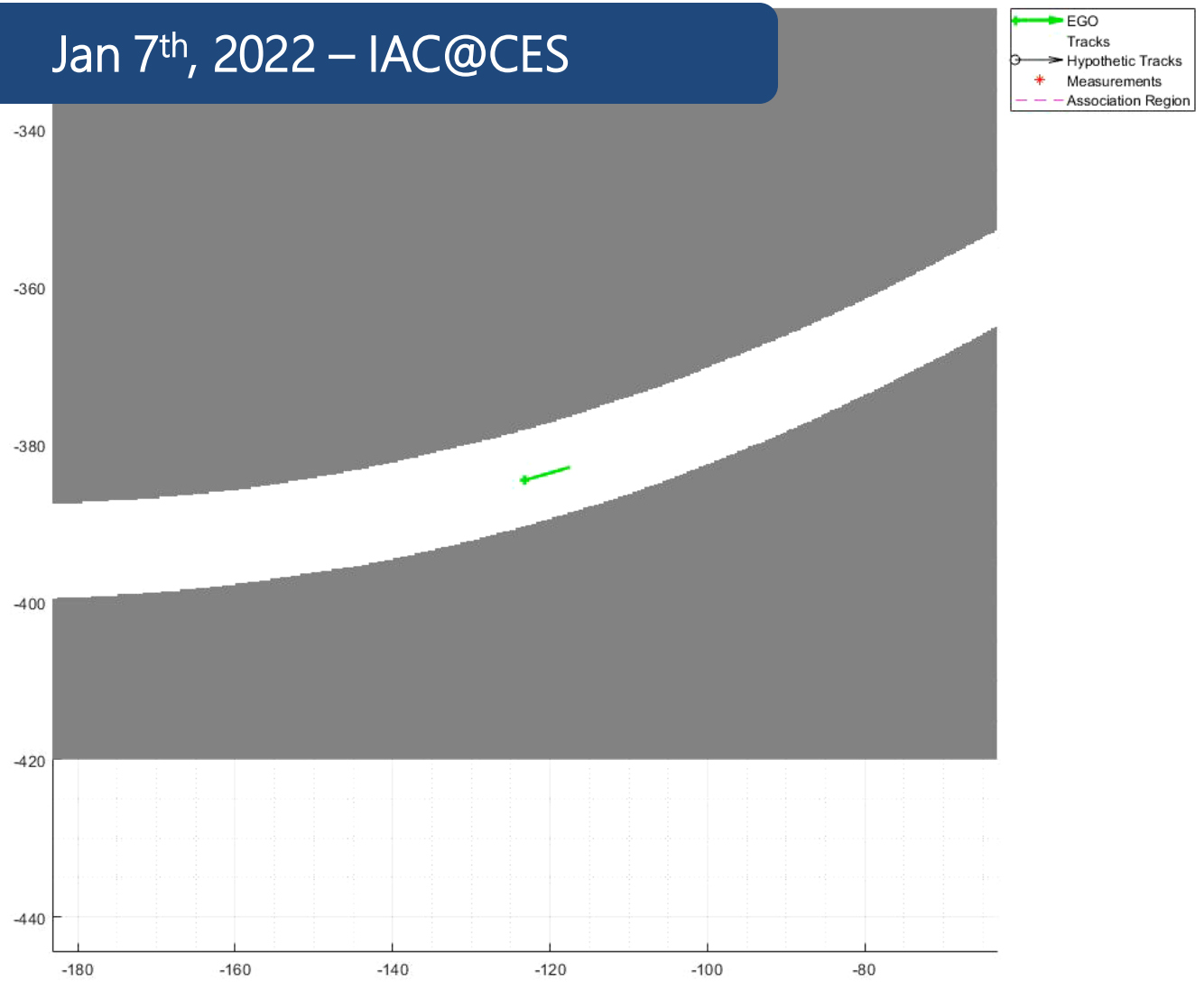
Measure

Measure association

Track management

Correction

Jan 7th, 2022 – IAC@CES



Target tracking in the race

Nov 11th, 2023 – IAC@TMS

